IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 16 March 2025, accepted 22 April 2025, date of publication 30 April 2025, date of current version 7 May 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3565980

RESEARCH ARTICLE

TEDVIL: Leveraging Transformer-Based Embeddings for Vulnerability Detection in Lifted Code

GARY A. MCCULLY[1], (Member, IEEE), JOHN D. HASTINGS[1], AND SHENGJIE XU[2], (Senior Member, IEEE)

[1]The Beacom College of Computer and Cyber Sciences, Dakota State University, Madison, SD 57042, USA
[2]College of Applied Science and Technology, The University of Arizona, Tucson, AZ 85721, USA

Corresponding author: Gary A. McCully (gary.mccully@ieee.org)

ABSTRACT Ransomware and other malware inflict devastating financial and operational damage on organizations worldwide by exploiting deeply embedded, hard-to-detect vulnerabilities in their systems. Detecting these vulnerabilities in compiled code before malicious actors exploit them remains a critical challenge in cybersecurity. This research introduces TEDVIL (**T**ransformer-based **E**mbeddings for **D**iscovering **V**ulnerabilities **i**n **L**ifted Code), a novel framework which uses transformer-based embeddings to train neural networks to detect vulnerabilities in lifted code. The framework was implemented using bidirectional (BERT and RoBERTa) and unidirectional (GPT-1 and GPT-2) transformer-based models to generate embeddings for training Long Short-Term Memory (LSTM) neural networks to detect stack-based buffer overflows in Low-Level Virtual Machine (LLVM) intermediate representation code. For comparison, simpler word2vec models (Skip-Gram and Continuous Bag of Words) were also trained, and their embeddings were used to train LSTMs. The results show that the LSTMs using GPT-2 embeddings outperformed those using GPT-1, BERT, RoBERTa, and word2vec embeddings, achieving a top accuracy of 92.5% and an F1-score of 89.7%. Notably, these results are achieved when the embedding model is trained with a dataset of just 48,000 functions, demonstrating effectiveness in resource-constrained settings. The findings underscore the effectiveness of TEDVIL in identifying hard-to-detect vulnerabilities in compiled code, and lay the groundwork for future research in leveraging transformer-based models for vulnerability detection.

INDEX TERMS Bidirectional encoder, binary security, buffer overflow, machine learning, neural network, transformer-based embedding, unidirectional encoder.

I. INTRODUCTION

Organizations rely heavily on third-party software to achieve their business objectives. For example, many organizations use Microsoft Windows operating systems across their laptop, desktop, and server environments [1]. Microsoft is closed-source, meaning they provide compiled binaries, but do not provide the source code to their customers [2], [3]. This creates challenges for organizations seeking to review software for source code vulnerabilities before deployment. During compilation from languages such as C/C++, high-level structures such as variable names, data structures, and code comments are lost, making the identification of vulnerabilities in compiled binaries significantly more resource intensive compared to source code [4]. Furthermore, the assembly code differs significantly across architectures, such as x86-64, MIPS, SPARC, and ARM. Thus, these organizations must trust the third party (e.g., Microsoft) to follow secure coding practices when developing their applications [5]. Unfortunately, like all software, coding mistakes can introduce bugs that threat actors exploit to cause mass destruction to organizations worldwide. For

The associate editor coordinating the review of this manuscript and approving it for publication was Tyson Brooks.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

example, several notable worms have exploited protocols in Microsoft's operating systems, including WannaCry [6], Sasser [7], Conficker [8], and Code Red [9]. Recovering from these types of attacks has cost organizations billions of dollars (USD) [10], [11], resulted in substantial losses in revenue [12], and even cost people their lives [13], [14]. To address these challenges, many researchers have turned to machine learning as a potential solution for identifying vulnerabilities in compiled binaries (Section VIII).

Researchers have investigated the application of Natural Language Processing (NLP) techniques in machine learning to the task of identifying vulnerabilities within source code, assembly code, and lifted code. The use of these techniques for this purpose is inspired by the observation that natural language and source code exhibit similarities. Thus, these researchers have used neural network architectures traditionally used for NLP tasks [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28] and explored embedding models commonly used to learn language-level semantics and transform words into numerical representations that preserve these semantics. One of the most used embedding models for this task in the current literature is word2vec [15], [18], [19], [22], [23], [25], [29]. Word2vec models produce static embeddings, meaning that once the model is trained, a given token always generates the exact vector representation, regardless of its surrounding tokens or position within the sequence. The weaknesses of this model are well known, and alternative embedding models inspired by or based on it have been proposed [20], [27], [28], [30], [31]. However, these alternatives lack the robustness of the transformer-based models introduced in [32].

Transformer-based models dynamically generate token embeddings based on the token's position within the sequence and surrounding context. Although these models have been studied in the context of code similarity tasks [33], [34], [35], [36], [37], they have only been used in a few studies to find specific vulnerabilities in compiled code [31], [38], [39]. Building on the authors' previous works [40], [41], this research aims to address this gap by introducing TEDVIL (***T**ransformer-based **E**mbeddings for **D**iscovering **V**ulnerabilities **i**n **L**ifted Code*), a novel framework which provides a model for using transformer-based embeddings to capture context in lifted code. The implementation of TEDVIL in this paper achieved high accuracy in vulnerability detection when using embeddings from custom-trained embedding models to train neural networks for detecting stack-based buffer overflows. Furthermore, it moves the conversation forward through the following contributions to the field:

- This research introduces TEDVIL, the first formal framework that utilizes transformer-based embeddings to identify categories of vulnerabilities (e.g., stack-based buffer overflows) in lifted code.
- This study is the first to train unidirectional transformer-based embedding models (i.e., GPT-1 and GPT-2) from scratch to generate contextual embeddings for identifying categories of vulnerabilities in lifted code (i.e., LLVM IR).
- The study demonstrates the feasibility and effectiveness of using relatively small datasets (48K LLVM functions) to train transformer-based models such as BERT, RoBERTa, GPT-1, and GPT-2 for vulnerability detection. This is significant given that large language models typically require extensive datasets, and it opens the door for applications in resource-constrained environments.
- The research provides a comparative analysis of embeddings from transformer-based models (BERT, RoBERTa, GPT-1, GPT-2) and traditional word2vec models (CBOW, Skip-Gram) for vulnerability detection. The findings offer valuable insights into the strengths and limitations of these approaches, guiding future efforts in embedding model selection for code vulnerability detection.
- The study identifies key challenges and opportunities for advancing vulnerability detection in compiled code, proposing specific avenues for future research, including exploring additional embedding models, alternative neural network architectures, varying dataset sample sizes for training models, and analyses using different Intermediate Representation (IR) languages.

The remainder of the paper is organized as follows. Section II introduces concepts and technologies vital to understanding the rest of the paper. Section III introduces the TEDVIL framework; this is a repeatable framework that can be used for discovering vulnerabilities in lifted code using embeddings from transformer-based models and is implemented in Section IV.

Section V provides the results of the implementation of TEDVIL using four transformer-based models and two simpler embedding models for comparison. Section VI highlights key findings from this study, while Section VII outlines potential directions for future research. Finally, Section VIII summarizes related works most applicable to this research project, and Section IX concludes this study.

II. BACKGROUND

A. LLVM IR

Low-Level Virtual Machine (LLVM) IR is a low-level, platform-independent intermediate code designed to represent the semantics of high-level programming languages [42]. It provides a mechanism for handling exceptions in high-level languages and includes instructions to support typed address arithmetic [43]. Furthermore, its platform independence makes LLVM IR particularly appealing for identifying patterns that may occur in software compiled for various architectures (e.g., x86-64, ARM, MIPS, SPARC). LLVM IR employs a Static Single Assignment (SSA) approach, where each variable is assigned exactly once and remains constant within its scope [44]. Listing 1 is a basic LLVM IR code sample that prints the words *LLVM Example* to the terminal using the `puts` method in the standard C library.

```
; LLVM Example

; Declare the string as a global constant
@llvmExample = constant [14 x i8] c"LLVM Example\0A\00"

; Declare puts function that is within the standard C library
declare i32 @puts(ptr)

; Definition of main function
define i32 @main() {

    ; Call puts function to write string
    call i32 @puts(ptr @llvmExample)

    ret i32 0
}
```

Listing 1. LLVM example.

B. STACK-BASED BUFFER OVERFLOWS

Stack-based buffer overflows are vulnerabilities that occur when a stack frame's return address is overwritten with an address controlled by a threat actor; this can result in a program crash or, in some cases, allow the attacker to hijack the program's control flow to execute malicious code [45, p. 200-217]. The mechanics of this vulnerability are well studied and relatively straightforward. When a function is called from another function, the program's control flow needs a way to return to the calling function after executing the called function. To achieve this, the parameters required to execute the called function are pushed to the top of the stack, followed by the return pointer, which stores the address of where to return once the called function has completed executing [45, p. 204]. The called function is given its own stack frame, which may include space for temporary data elements that users could directly or indirectly control. The stack grows toward the return pointer, and if the data is not securely copied to the stack, the amount of data written may exceed the program's expectations [45, p. 204]. This can lead to overflowing data and overwriting the return address or a portion of it [45, p. 204].

Table 1 visually represents what a stack frame may look like before and after a stack-based buffer overflow has occurred. In this case, a user could write to a location on the stack an amount of data greater than the program expected; the user wrote a series of A's until the return address was written. At this point, the attacker overwrote this return address with the address containing the shellcode.

Listing 2 illustrates a simple stack-based buffer overflow example written in LLVM IR. In this example, the string is 50 bytes long, whereas the buffer designated to store the string is only 20 bytes. The call to `strcpy` copies the string into the buffer without checking for size differences. Consequently, the buffer overflows and overwrites adjacent memory structures on the stack, including the return address. When run in a debugger on a system with an x64 architecture,

TABLE 1. Buffer overflow example (x86 Architecture).

Address	Function Stack	Post Exploitation
0xFFFFFFFF	Top of Stack	
0xDDCCBB04	Stack Space User Can Write	Shellcode
0xCCBBCC00	Return Address (EIP)	0xDDCCBB04
0xCCBBCBFC	Stack Frame Pointer	AAAA
0xCCBBCBF8		AAAA
0xCCBBCBF4		AAAA
0xCCBBCBF0		AAAA
0xCCBBCBEC		AAAA
0xCCBBCBE8	Writable Variables	AAAA

this code shows the return address is overwritten with the value $0 \times 4141414141414141$, corresponding to eight consecutive 'A' characters in ASCII. This results in a segmentation fault and causes the program to crash. This type of vulnerability is identified as CWE-121 in the Common Weakness Enumeration (CWE) database [46].

C. EMBEDDING MODELS

1) WORD2VEC: CBOW AND SKIP-GRAM

The word2vec model, introduced in 2013, was designed to learn simple semantic relationships between words in natural languages; it employs two techniques, Continuous Bag of Words (CBOW) and Skip-Gram, to capture word-level relationships [47]. The CBOW technique learns token-level semantics by predicting a missing word in a sequence based on the surrounding words [47]. For example, with the CBOW approach, the model might be trained using a sentence such as *The bird ____ a beautiful song*, where it may predict a word like *chirped*. If the model predicts an incorrect word, it is updated to improve its ability to predict the correct word in future iterations. However, the Skip-Gram approach attempts to learn the surrounding context words for a given target word [47]. In the bird example, the target word *chirped* would be used to predict its neighboring words, such as *the*, *bird*, *a*, *beautiful*, and *song*. One task at which word2vec models excel is in learning similarities between words. For example, the word *toast* in the sentence *We are having french toast for dinner* could easily be replaced with *fries* or *bread* and still make perfect sense. In this case, word2vec would learn the semantic similarities in these words, and generate embeddings that capture these relationships.

2) BERT & ROBERTA

BERT, short for **B**idirectional **E**ncoder **R**epresentations from **T**ransformers, was introduced in October 2018 and is one of the first bidirectional transformer-based embedding models [48]. Bidirectional models, like BERT, learn token embeddings by leveraging context from both directions through a masking process used during training [48]. Earlier transformer-based models were primarily unidirectional, but BERT's bidirectional approach achieved higher scores across various NLP benchmarks compared to its predecessors [48]. During training, the model processes token sequences,

```
; Stack-Based Buffer Overflow Example

; Declare the string as a global constant.
@BufferOverflow = private constant [50 x i8] c"
    AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA\0A\00"

; Declare strcpy function that is within the standard C library
declare ptr @strcpy(ptr, ptr)

; Definition of main function
define void @main() {

    ; Define a buffer to store the @BufferOverflow string as array of 20 bytes
    %buffer = alloca [20 x i8]

    ; Call strcpy function to copy the @BufferOverflow string into the allocated
        buffer.
    call ptr @strcpy(ptr %buffer, ptr @BufferOverflow)

    ret void
}
```

Listing 2. Stack-based buffer overflow example.

randomly masks a percentage of the tokens, and learns to predict them based on surrounding context [48].

RoBERTa, short for **R**obustly **O**ptimized **BERT** Pre-training **A**pproach, was introduced in July 2019 as an enhancement to BERT [49]. One of the key differences lies in the masking strategy. BERT uses static masking, where the same tokens are masked across all epochs, while RoBERTa employs dynamic masking, randomly selecting tokens for masking in each epoch [49]. This dynamic approach increases variability in training data, resulting in richer and more contextualized embeddings.

BERT and RoBERTa models are classified as transformer-based embedding models and use architectures based on [32]. Unlike static embeddings such as word2vec, transformer-based embeddings are context-aware. This means they consider both the token's position in the sequence (via positional encodings) and the surrounding tokens when generating its embedding. As a result, the same token may result in different embeddings depending on its context. For example, Table 2 highlights the differences between these two approaches to embedding generation. This table shows that the static embedding generates the same value for the word *watch*. However, the first instance of this word is used as a verb, and in the second case, it is used as a noun. The transformer-based embedding can identify this difference and generate a different embedding for both cases of how the word is used.

TABLE 2. Static vs. Transformer-based embeddings.

Static Embeddings (i.e. Skip-Gram & CBOW)				
[0,0,0]	[0,0,1]	[0,1,0]	[0,0,0]	[0,1,1]
Watch	*how*	*the*	*watch*	*stopped*
Transformer-Based Embeddings (i.e. BERT, RoBERTa, GPT-1, & GPT-2)				
[0,0,0]	[0,0,1]	[0,1,0]	[0,1,1]	[1,0,0]
Watch	*how*	*the*	*watch*	*stopped*

3) GPT-1 & GPT-2

Released in 2018, OpenAI introduced the **G**enerative **P**re-trained **T**ransformer (GPT) model; unlike models that use masking strategies during training, GPT was pretrained on a large corpus of unlabeled text, which was then fine-tuned to perform specific tasks [50]. In February 2019, the second iteration of this model, GPT-2, was released; it was trained on a much larger dataset, consisting of 8 million documents and 1.5 billion parameters [51]. Beyond the increase in training data, there are notable architectural differences between the two models, including adjustments to layer normalization and weight scaling [51].

This study uses these models to generate LLVM IR code-level embeddings. Like BERT and RoBERTa, GPT-1 and GPT-2 are transformer-based embedding models that leverage token positions and surrounding words to determine context [50]. However, unlike BERT and RoBERTa, GPT embeddings are unidirectional, considering only the words to the left of the target token when generating its embedding [48].

D. LSTM NEURAL NETWORKS

LSTM neural networks are a subset of Recurrent Neural Networks (RNN) [52]. Unlike feedforward neural networks, where information flows strictly from the input to the output, RNNs combine the hidden state of one step with the input from the next step. This concept is illustrated in Figs. 1 and 2. This recurrent connection allows the network to retain and use information from previous steps in the sequence, enabling it to build on past outputs and hidden states to predict future

states in the sequence [52]. For example, Table 3 shows how an RNN may change word predictions as the input changes dynamically from the user. An LSTM implements this RNN functionality through a series of gates, allowing it to retain essential information while discarding irrelevant details [52].

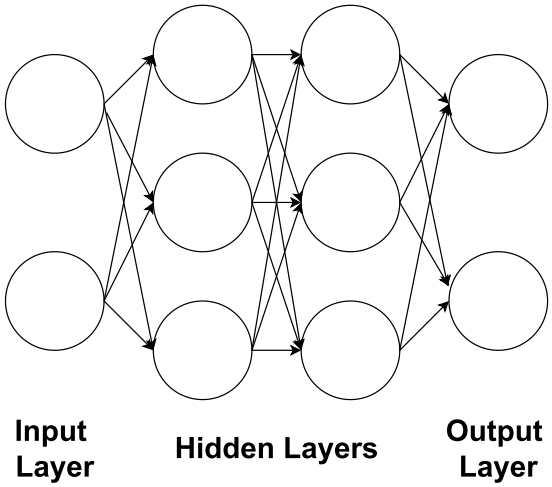


FIGURE 1. Feedforward neural network.

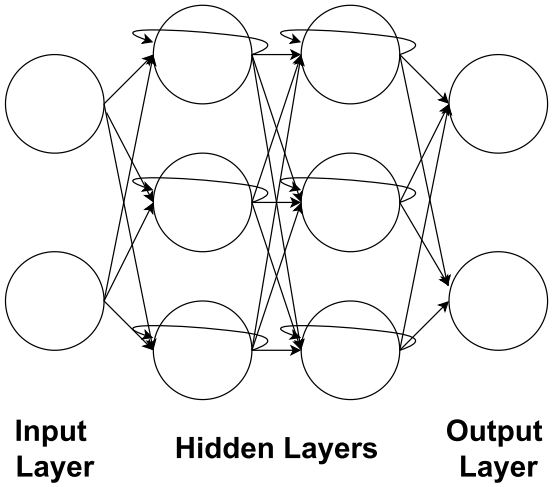


FIGURE 2. Recurrent neural network.

TABLE 3. RNN example with input as green and predicted next word in red.

Time Sequence	Text Generation					
t	I	like	ice	cream		
t+1	I	like	ice	cold	water	
t+2	I	like	ice	cold	lemon	ade

III. FRAMEWORK: TEDVIL

TEDVIL, standing for ***T**ransformer-based **E**mbeddings for **D**iscovering **V**ulnerabilities **i**n **L**ifted Code*, introduces a systematic approach to detect vulnerabilities in lifted code using embeddings from transformer-based models. This section describes the key components of TEDVIL, illustrated in Figure 3. Furthermore, Section IV presents details of an implementation of this framework, focusing on the evaluation of transformer-based unidirectional (GPT-1 and GPT-2) and bidirectional (BERT and RoBERTa) embeddings.

A. SOURCE CODE DATASET SELECTION

1) EMBEDDING MODEL DATASET

This dataset should consist of valid source code that can be successfully compiled. The code will be compiled and lifted into IR to train a custom transformer-based embedding model (e.g., BERT, GPT) to capture the code's semantics. Therefore, the code must be sufficiently complex to effectively convey its semantic richness. Although transformer-based embedding models are typically trained on millions of samples, as highlighted in Section II-C, the implementation described in Section IV achieved strong performance with a dataset of only approximately 48,000 samples. Furthermore, the training samples for these embedding models only need valid representations of the code used to teach the model code-level semantics. These samples can be generated by compiling and lifting legitimate software projects, such as open-source projects.

2) NEURAL NETWORK MODEL DATASET

Although this framework focuses on discovering vulnerabilities in lifted code, it achieves this by selecting a source code dataset, compiling the samples, and applying techniques to lift the compiled code. The first step in this process involves selecting a high-quality source-code-based dataset that includes the classification of the vulnerabilities to be identified. For example, the National Institute of Standards and Technology's (NIST) Software Assurance Reference Dataset (SARD) [53] project provides several test suites of source code samples, encompassing various classifications of vulnerabilities such as insecure direct object references, local file inclusion, and command injection. Although the implementation of TEDVIL in this paper utilizes the Juliet 1.3 dataset, other datasets could be substituted as alternatives. Here are a few considerations when selecting a source-code dataset:

- Does the dataset include the classification of the vulnerability that the model aims to identify?
- Does the dataset provide sufficient samples of this vulnerability classification to train a model to detect the vulnerability?
- Are the source-code samples provided in a format that can be easily compiled into machine code?
- Are the functions named in a way that helps identify where the vulnerability occurs within the code?

B. COMPILATION AND LIFTING

Once each source code-based dataset is selected, the source code samples must be converted into compiled code, either as object files or fully executable binaries. It is also essential to understand the compilation flags used by the selected

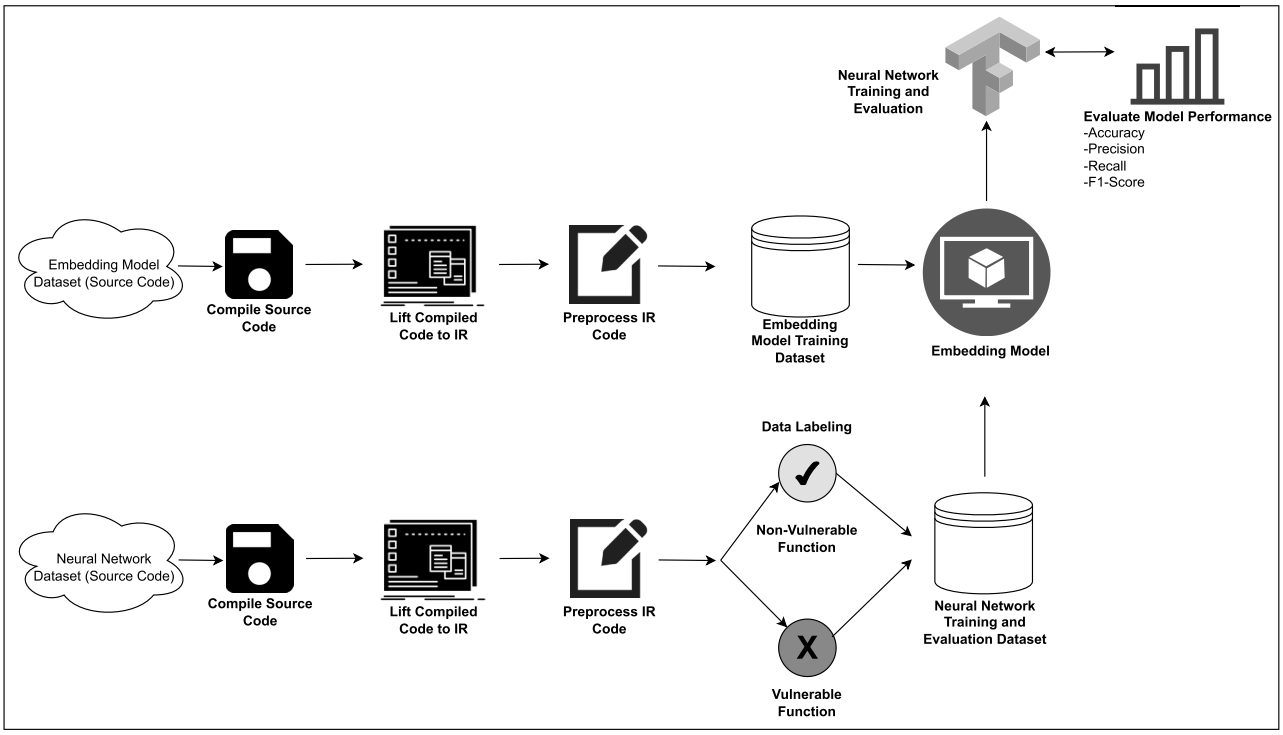


FIGURE 3. TEDVIL framework.

compiler and the potential impact these flags may have on the compiled code.

Once the source code samples have been compiled, the binaries will be lifted to an IR. Some IR languages include VEX [54], Binary Analysis Platform (BAP) [55], P-Code [56], and LLVM [42]. Various tools can be used to lift the binaries to these IR representations, including angr (VEX IR) [57], BAP (BAP IR) [57], Ghidra (P-Code) [56], and RetDec (LLVM IR) [58]. One of the advantages of using an IR over assembly instructions is that an IR is a cross-architecture language. This means it can take assembly code from different architectures (e.g., ARM, x86-64, etc.) and lift it into a uniform representation consistent across all architectures.[1]

C. PREPROCESSING

The purpose of preprocessing is to eliminate patterns specific to a single code sample without affecting its functionality. The goal of an effective machine learning model is to generalize from specific examples to broader patterns. Rather than training the model to classify whether a specific sample, such as sample **XYZ**, is vulnerable or not, it is preferable for the model to learn the patterns in sample **XYZ** that result in a vulnerability. Thus, data specific to a particular sample that does not affect its functionality should be flattened into a more generalized token sequence. A good example of this kind of unique function-specific pattern would be a function name. A function name is unique to a program and specifies a set of instructions executed when the function is called. However, the actual name of the function does not affect its behavior. For example, a function named `A` that prints `Hello World` to the console will perform the same operation even if renamed to `B`. Therefore, a common preprocessing step is to standardize function names, such as renaming `Uniquely_Named_Function` to `Function_One`.

[1]It is important to note that this compilation and lifting process will be applied to both source-code datasets.

D. PREPROCESSED DATASET CREATION

1) EMBEDDING MODEL DATASET

Once these code samples have been preprocessed, they are ready to train the embedding models. The remaining step in this process is to ensure they are in the correct format (e.g., CSV, TXT) for training the embedding model.

2) NEURAL NETWORK MODEL DATASET

The second dataset will be used to train neural networks to discover vulnerabilities in the lifted code. Thus, these code samples must track which samples contain the vulnerability and which do not. In many cases, such as the Juliet dataset, the function names can be used to determine which functions contain the vulnerability. The implementation described later in this study achieved strong results using a dataset of approximately 4,000 samples.

E. EMBEDDING MODEL TRAINING

There are several choices of transformer-based embedding models that can be selected, including BERT [48],

RoBERTa [49], DistilBERT, GPT [50], GPT-2 [51], XLNet [59], ALBERT [60], ELECTRA [61], DeBERTa [62], Longformer [63], and BigBird [64]. Each of these models possesses architectural features that may produce interesting or unique characteristics when generating embeddings to capture the semantics of the IR. This list is not exhaustive, and it remains to be seen which transformer-based models would generate the most effective embeddings for representing the semantics of a specific IR suitable for training a neural network.

F. NEURAL NETWORK TRAINING

The last step in TEDVIL is to train neural networks to identify vulnerable and non-vulnerable IR code segments. This process involves dividing the dataset used to train and test neural networks into distinct subsets for training, testing, and, optionally, hyperparameter tuning. The training process involves providing IR code samples to the embedding models that generate embeddings. These embeddings are passed to the neural network to teach it to differentiate between vulnerable and non-vulnerable code. After each epoch, the network is evaluated using embeddings from the testing subset to assess its performance at that stage.

Research projects can use several types of neural networks for this task. For example, various Recurrent Neural Networks (RNNs) have been used to discover vulnerabilities in compiled code, including LSTM, Bidirectional LSTM (BLSTM), Gated Recurrent Unit (GRU), and Bidirectional GRU (BGRU), as mentioned in Section VIII. Furthermore, several other types of neural networks may be used in the vulnerability discovery process, including Feedforward Neural Networks (FNN), Message Passing Neural Networks (MPNN), Graph Neural Networks (GNN), and Convolutional Neural Networks (CNN).

G. NEURAL NETWORK EVALUATION

Several metrics can be used to evaluate the performance of the model. Four of the most common scores used to evaluate neural networks are accuracy [align 1], precision [align 2], recall [align 3], and F1-score [align 4]. In the following equations, TP stands for true positive, TN stands for true negative, FP stands for false positive, and FN stands for false negative.

- **Accuracy**: This metric measures the ability of a model to correctly classify whether a given sample contains a vulnerability [65, p. 289]. The formula for accuracy is as follows:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}. \tag{1}$$

- **Precision**: This metric measures the proportion of samples accurately classified as vulnerable out of all samples that were predicted to be vulnerable; it is particularly important for assessing how well the model minimizes false positives [65, p. 289]. The following equation provides the formal definition of this metric:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}. \tag{2}$$

- **Recall**: This metric measures the proportion of correctly classified vulnerable samples relative to the total number of actual vulnerable samples; this metric is especially significant when determining a model's ability to limit false negatives [65, p. 289]. The recall formula is defined as follows:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \tag{3}$$

- **F1**: This fourth metric combines precision and recall into a single formula and is defined as follows [66, p. 93]:

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{4}$$

IV. IMPLEMENTATION

This section demonstrates how the framework can be implemented in practice. While TEDVIL focuses primarily on generating embeddings with transformer-based models, the implementation in this section also incorporates word2vec: Skip-Gram and CBOW models for comparative reasons. These simpler models serve as effective baselines for comparison with the more complex transformer-based approaches. As illustrated in Fig. 4, the steps followed to implement TEDVIL included:

1) Samples from the Juliet dataset were compiled into object files (Section IV-A).
2) The RetDec tool was used to lift the object files to LLVM IR (Section IV-B).
3) A series of preprocessing steps was applied to each LLVM IR file (Section IV-C).
4) Datasets were created for training embedding models and neural networks (Sections IV-D and IV-E).
5) The dataset used to train the embedding model was utilized to train the following models (Section IV-F):
   - word2vec: CBOW embedding
   - word2vec: Skip-Gram embedding
   - BERT
   - RoBERTa
   - GPT-1
   - GPT-2
6) Several LSTMs were created using the embeddings from these models (Section IV-G).
7) LSTM model performance was compared using accuracy, precision, recall, and F1-score metrics (Section IV-H).

The following subsections describe these steps in more detail.

A. DATASET SELECTION

1) EMBEDDING MODEL DATASET

The source code dataset used to train the embedding models consisted of C/C++ samples from the Juliet 1.3 dataset. This

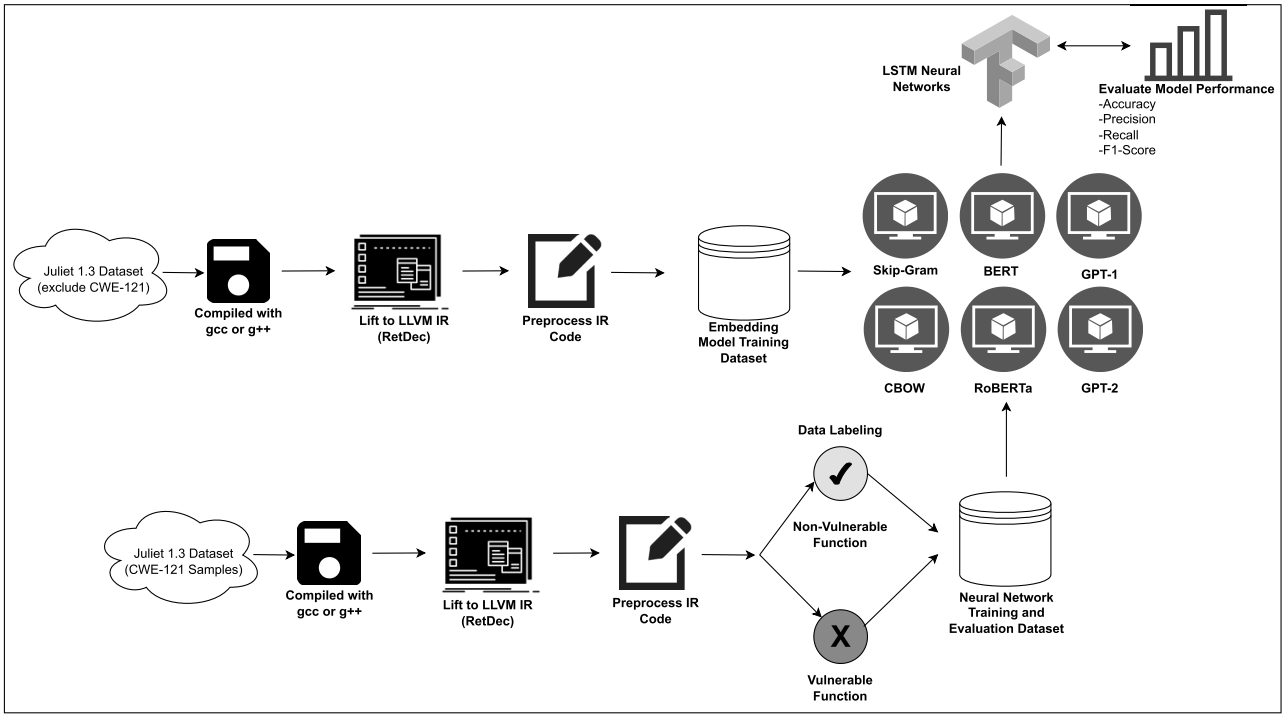


FIGURE 4. Building embedding models using lifted code.

dataset includes code samples that demonstrate vulnerabilities across several vulnerability classifications, along with variants where these vulnerabilities have been remediated. Examples of these classifications include categories such as use-after-free, null pointer dereference, and command injection vulnerabilities. In most cases, the dataset contains multiple samples with different variants of the same vulnerability classification. This makes it well-suited for training models to detect such vulnerabilities and has been extensively used for this purpose (Section VIII).

The dataset used to build the embedding models consisted of samples from the Juliet dataset, excluding those with CWE-121 vulnerabilities. These samples were chosen for their structural complexity, making them well-suited for capturing patterns in the LLVM IR language. Additionally, the dataset's manageable size made it ideal for testing TEDVIL in resource-constrained environments.

2) NEURAL NETWORK MODEL DATASET

The source code samples used to train the neural networks were the C/C++ samples from the Juliet dataset that contained CWE-121 vulnerabilities.

B. COMPILE AND LIFT CODE SAMPLES

Code samples from the Juliet dataset were compiled into object files on a Kali Linux 2023.3 system using gcc/g++ and lifted to LLVM IR with the RetDec tool [58]. The samples were also compiled on a Windows 10 system; however, these samples were excluded from the dataset after analysis showed that calls to the standard C library were stripped during the lifting process.

C. LLVM PREPROCESSING

This study applied the preprocessing steps outlined in Table 4 to all lifted LLVM functions in both LLVM IR datasets. One of the initial steps in this process was renaming unique function names within LLVM to a more generalized form that focused on function order. For example, functions containing vulnerabilities included the word *bad* in their names. This naming convention could unintentionally leak information about vulnerable samples to the model. As a result, rather than detecting code patterns that lead to vulnerabilities, the model could instead learn to differentiate based on the presence of the word *bad*. To mitigate this issue, functions were renamed from their unique identifiers to generic names such as func_one, thereby minimizing the risk of leaking information about function capabilities through their names.

A second preprocessing step involved replacing special characters and numbers with their corresponding word representations. This ensured that any tokenizer used would treat these tokens similarly to words. The rationale behind this approach was that special characters function differently in natural language compared to code. In natural languages, words serve as the primary medium of communication, whereas in most programming languages, meaning is conveyed mainly through a combination of a limited set of keywords and frequent use of special characters.

Another preprocessing step involved breaking down numerical sequences into their individual digits. For example, the number *1068* would be represented as *one zero six eight*. The rationale behind this approach is that while specific numerical sequences like *1068* may be rare in code, individual digits and their relative positions are more common. By representing numbers in this way, embedding models can learn general numerical relationships within the code rather than memorizing exact numeric sequences.

TABLE 4. Preprocessing steps.

Action	Before	After
LLVM functions created within the code were given a unique identifier	@_func_unique_name()	func_x ()
Function names that were not defined, but were called within the code were given a unique identifier	@_undefined_func_unique_name()	extfuncvar_x ()
global variables, labels, stack pointers, and numeric metadata were renamed to generic tokens	@global_var_offset dec_label_pc_offset: %stack_var_offset !123	globalvar_x declabel_x stackvar_x !x
Digits were isolated from the surrounding characters and converted into their word forms	123	one two three
Special characters were renamed to text-based tokens	{ . ,	opcurl dotmark commark

[a] x is a number used to track the order in which it was defined.

D. EMBEDDING DATASET CREATION

The embedding dataset was composed of the functions from the LLVM IR samples that did not contain stack-based buffer overflows. Although the samples in this dataset contained samples from several classes of vulnerabilities as well as the fixed varlets, the vulnerable status of these samples was not tracked since these samples were only used by the embedding models to learn the semantics of the LLVM IR language. All samples were stored in a text file, with one function per line. While there were 457,529 functions in total, only 48,157 remained after duplicate removal. Table 5 breaks these numbers down.

E. NEURAL NETWORK DATASET CREATION

The remaining preprocessed LLVM IR samples from the Juliet dataset (i.e. those containing CWE-121 vulnerabilities) were placed in a dataset used to train neural networks to detect code samples containing stack-based buffer overflows. Although the dataset initially contained 12,069 samples, it was reduced to 3,901 samples after removing duplicates from the preprocessed functions. Manual analysis of the duplicate code revealed that several functions provided minimal functionality, such as calling secondary functions containing vulnerabilities. Additionally, samples with more than 2,048 tokens were excluded due to resource limitations on the systems used to train the neural networks. This further reduced the dataset to 3,802 samples. The loss of 99 samples was considered acceptable, as retaining them would have introduced unnecessary complexity into the implementation. A breakdown of this dataset is in Table 5.

TABLE 5. Embedding and LSTM training datasets.

Dataset Purpose	Training (size) LLM	Training (size) LSTM
Object Files	61,023	4,949
LLVM Files	61,023	4,949
Extracted Functions:		
Pre Duplicate Removal	457,529	12,069 Clean: 7,011 Vuln: 5,058
Post Duplicate Removal	48,157	3,901 Clean: 2,452 Vuln: 1,449
Post Removal of Functions More than 2048 Tokens	NA	3,802 Clean: 2,386 Vuln: 1,416

F. EMBEDDING MODEL TRAINING

The decision to focus on BERT, RoBERTa, GPT-1, and GPT-2 was based on their significance as some of the first transformer-based embedding models and their second-generation variants. Evaluating these foundational models provides a solid starting point for conducting a comparative analysis of transformer-based embedding models. Furthermore, the CBOW and Skip-Gram models are useful baselines for comparison with the more complex transformer-based embedding models, as they have been extensively used in prior studies to identify vulnerabilities in compiled binaries (Section VIII). This section details the specific implementation choices for each of these models.

There are some aspects of the transformer-based modes that remained constant among all models:

- HuggingFace's transformers library (v4.43.3) was used to build all embedding models (BERT, RoBERTa, GPT-1, and GPT-2).
- HuggingFace's tokenizers library (v0.19.1) was used for all tokenization processes used to train these models.
- The maximum size of the LLVM IR functions was limited to a token length of 2048.
- All embedding models consist of 12 hidden layers, each with 100 nodes and ten attention heads.
- The dataset used to train the embedding models was split into two subsets: 90% for training and 10% for validation.
- The loss from the training and validation steps reported every 1,000 steps, and training occurred over the course of 20 epochs.
- All training occurred on an Ubuntu system running dual NVIDIA TI 4090 GPUs.

1) WORD2VEC: CBOW AND SKIP-GRAM MODEL CREATION

The Gensim (4.3.0) library was used to train the CBOW and Skip-Gram embedding models, and the nltk (3.8.1) library's word tokenizer was used to tokenize LLVM IR tokens. The models were built using a dataset of preprocessed LLVM IR

samples from the Juliet dataset. For both the CBOW and Skip-Gram models, the output embedding vector size for each token was set to 100, the window size to five, and the minimum token count to one. After training, the model identified a total of 332 unique tokens among the LLVM IR samples used to train these models.

2) BERT AND ROBERTA MODEL CREATION

The `BertForMaskedLM` class was used to train a custom BERT model, with LLVM IR samples tokenized using the `BertWordPieceTokenizer` class. Similarly, the `RobertaForMaskedLM` class was employed to build a RoBERTa model, utilizing the `ByteLevelBPETokenizer` class for tokenization. For both the BERT and RoBERTa models, the best validation loss occurred after step 54,000, as shown in Table 6.

TABLE 6. BERT/RoBERTa five best loss scores by validation step.

	BERT		RoBERTa	
Step	Training	Validation	Training	Validation
50,000	2.555300	2.496149	2.558400	2.494601
51,000	2.555500	2.497498	2.557400	2.488815
52,000	2.551600	2.493027	2.559900	2.485950
53,000	2.546700	2.495847	2.548000	2.487699
54,000	2.554400	2.492988	2.557500	2.483047

3) GPT-1 AND GPT-2 MODEL CREATION

The `OpenAIGPTLMHeadModel` class from the transformers library was used to create a GPT-1 model. The BERT BasicTokenizer was employed to tokenize LLVM IR tokens as a fallback tokenizer, which the transformers library uses when dependency issues prevent using the `OpenAIGPTTokenizer`. The library used to train the model did not support multiple GPUs, so a single TI 4090 was used for training. Thus, the training steps were doubled compared to those of the BERT, RoBERTa, and GPT-2 models, as only one GPU could be utilized. Table 7 presents the loss scores for the last five thousand steps. The embedding model generated at step 108,000 was used for generating embeddings, since it had the lowest validation loss score. This choice was made because it aligns with the criteria used to select the best-performing embedding model during the training of other transformer-based embedding models. However, step 108,000 did not yield the best score on the training dataset. Thus, although the model at this step was used for generating embeddings to train neural networks, it is reasonable to suggest that the model generated at step 107,000 may have been a better choice, as both the training and validation scores were lower at that step.

The `GPT2LMHeadModel` class was used to build the GPT-2 model and the `ByteLevelBPETokenizer` class was used to tokenize the LLVM IR code. Unlike the GPT-1 model, no issues arose during training with both GPUs. The model showed steady improvements in accuracy every 1,000 steps, as shown in Table 8, with the validation set achieving its lowest loss score at step 54,000.

TABLE 7. GPT-1 five best loss scores by validation step.

Step	Training Set	Validation Step
104000	0.065700	0.052784
105000	0.065500	0.052687
106000	0.065900	0.052701
107000	0.065300	0.052600
108000	0.066400	0.052555

TABLE 8. GPT-2 five best loss scores by validation step.

Step	Training Set	Validation Step
50,000	0.164600	0.132824
51,000	0.163700	0.132133
52,000	0.162800	0.132173
53,000	0.162100	0.131591
54,000	0.163200	0.131189

G. LSTM NEURAL NETWORK CONFIGURATION

Several LSTM neural networks were created during this implementation of TEDVIL. The neural network architecture included an input layer that received embeddings generated by the respective embedding models. These embeddings were passed through two LSTM layers, each containing 128 neurons. The Leaky Rectified Linear Unit (Leaky ReLU) activation function was applied to these layers, with a dropout rate of 20% used between them to mitigate overfitting. This falls within the recommended range of 20–30% for recurrent neural networks, as suggested by [66, p. 365]. The second LSTM layer passed its output to a single dense neuron with a sigmoid activation function, which determined whether the sample contained a buffer overflow. The dataset described in Section IV-E was split into training (80%) and testing (20%) subsets to train and evaluate the model's performance. Each LSTM was trained over 50 epochs.

Several LSTM models were trained using embeddings generated by each embedding model. These models were optimized using both Adam and SGD optimizers, with variations in the hyperparameters. The Adam optimizer was selected due to its proven success in [18] and [23]. However, as discussed in Section V, when using this optimizer, neural networks trained with embeddings from transformer-based models struggled to achieve better accuracy than simply assigning all samples to a single category. Thus, the simpler SGD optimizer was used as a secondary optimizer to train neural networks, leading to significant performance improvements in models trained with transformer-based embeddings.

Additionally, each model was trained twice, once with frozen embedding layers and once with unfrozen layers. The hyperparameters used for the LSTMs in this study are as follows:

- Adam optimizer: learning rates of 0.01, 0.001, and 0.0001
- SGD optimizer: a learning rate and momentum of 0.01
- SGD optimizer: a learning rate of 0.0001 and momentums of 0.01, 0.001, and 0.0001.

The learning rates for the Adam optimizer were based on Keras' default value (0.001). The models were trained to explore variations using the default learning rate and

values of an order of magnitude higher (0.01) and lower (0.0001). The primary learning rate for the SGD optimizer was intentionally set at 0.0001, one order of magnitude below the default Keras value. This decision was made due to the instability and lack of convergence observed with the Adam optimizer. Consequently, momentum [66, p. 352] was the primary hyperparameter explored with the SGD optimizer, focusing on low values to ensure stable updates and reduce noise sensitivity. Finally, a model was trained using the SGD optimizer with a higher learning rate (0.01) and a low momentum of 0.01. This final model serves as a point of comparison with models trained using more controlled gradients.

H. MODEL EVALUATION CRITERIA

LSTM neural networks were evaluated using accuracy, precision, recall, and F1-score, as detailed in Section III-G. Accuracy alone is not a strong indicator of model performance. For example, if a model had high accuracy but poor precision, someone responsible for locating and remediating vulnerabilities would struggle to make progress due to the many false positives they would need to sift through [65, p. 289]. Likewise, poor recall would be problematic because the model would fail to achieve its objective of identifying vulnerable code samples [65, p. 289]. However, the F1-score is a strong indicator of model performance, as it balances precision and recall, providing a more comprehensive measure of the model's ability to minimize false positives while effectively identifying vulnerabilities [65, p. 290].

V. RESULTS

The tables in this section exclude results from models that only achieved accuracy within the baseline of assigning all samples to a single category.

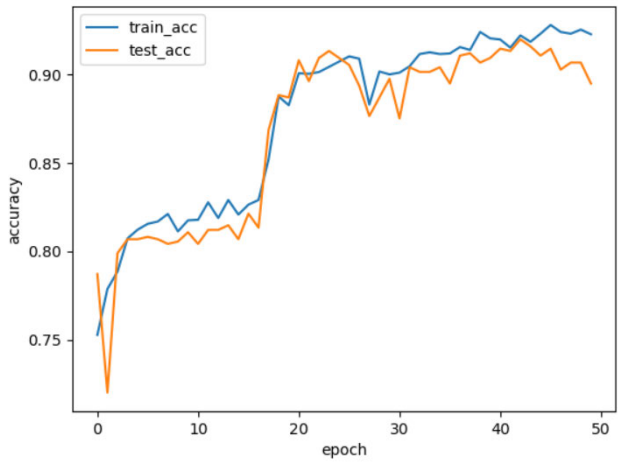


FIGURE 5. Accuracy graph of LSTM using Skip-Gram Embeddings with Adam Optimizer and learning rate 0.001(Unfrozen Embeddings).

A. WORD2VEC: SKIP-GRAM METRICS

Table 9 presents the results obtained when the embedding model was trainable, while Table 10 shows the results when the embedding model was frozen. The highest performance using word2Vec Skip-Gram embeddings was achieved with a trainable embedding model, which yielded an accuracy of 92.0% and an F1-score of 89.6%. This result was obtained with the Adam optimizer configured with a learning rate of 0.001.

The accuracy graph of the best-performing model (Fig. 5) reveals a notable pattern, including a significant gain at epoch 17. This pattern closely mirrors the findings of [67], which conducted a similar experiment with word2vec embeddings. However, this feature is absent (Fig. 6) when the same hyperparameters are chosen, and the embedding model is not trainable. The results using the SGD optimizer with learning rates of 0.0001 and momentum of 0.01, 0.001, and 0.0001 struggled to improve accuracy beyond the baseline of assigning all samples to a single category. However, as shown in the loss graph (Fig. 7), these models consistently achieved lower loss values with each epoch, suggesting that additional training epochs could eventually improve their performance.

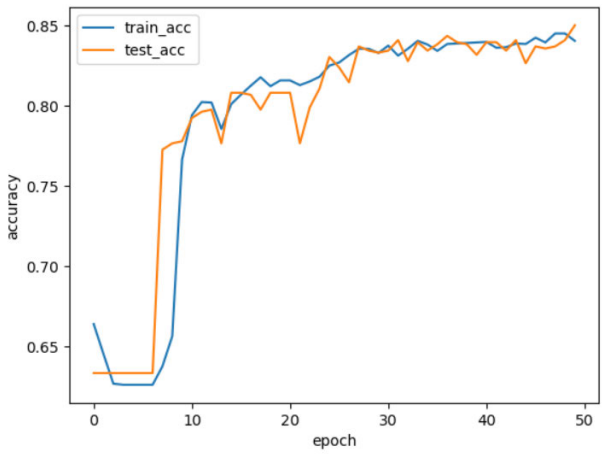


FIGURE 6. Accuracy graph of LSTM using Skip-Gram embeddings with adam optimizer and learning rate 0.001(Frozen Embeddings).

B. WORD2VEC: CBOW METRICS

The metrics presented in Tables 11 (Unfrozen Layers) and 12 (Frozen layers) were recorded where each model achieved its highest accuracy. Similarly to models that used Skip-Gram embeddings, the highest accuracy was achieved with a trainable embedding model and the Adam optimizer configured with a learning rate of 0.001; this model achieved an accuracy of 87.5% with an F1-score of 84.3%. Graph 8 demonstrates that the highest accuracy occurred at epoch 35. Like LSTMs trained with Skip-Gram embeddings, those trained with CBOW embeddings using the SGD optimizer also exhibited poor performance. However, identical to Skip-Gram embeddings, consistent improvements in loss scores across epochs suggest that additional training could have further enhanced performance.

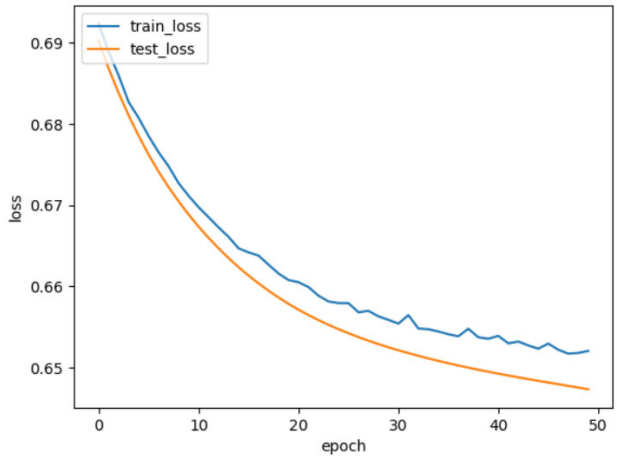


FIGURE 7. Loss graph of LSTM using Skip-Gram embeddings with SGD optimizer, learning rate 0.0001 and momentum 0.01 (Unfrozen Embeddings).

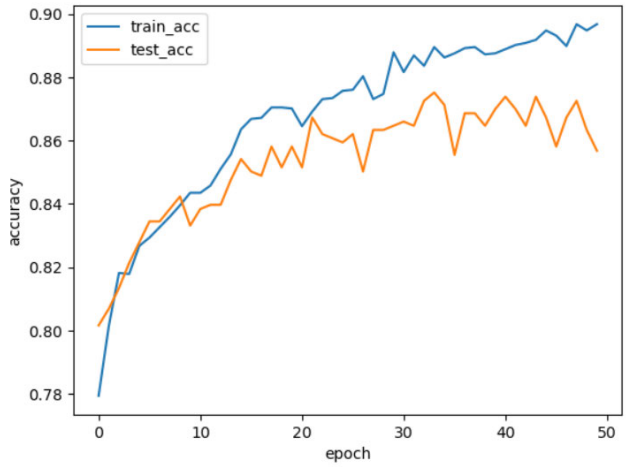


FIGURE 8. Accuracy graph of LSTM using CBOW embeddings with Adam optimizer and learning rate 0.001(Unfrozen Embeddings).

TABLE 9. word2vec: Skip-Gram metrics (Unfrozen Layers).

Hyper-parameters	Epoch	Loss	Accuracy	Precision	Recall	F1-Score
SGD -LR: 0.01 -Mom: 0.01	45	0.3663	80.9%	68.2%	90.0%	77.6%
Adam -LR: 0.01	35	0.4127	81.1%	69.1%	87.5%	77.2%
Adam -LR: 0.001	43	0.1848	92.0%	85.2%	94.6%	89.6%
Adam -LR: 0.0001	48	0.3282	84.0%	72.1%	91.8%	80.8%

TABLE 10. word2vec: Skip-Gram metrics (Frozen Layers).

Hyper-parameters	Epoch	Loss	Accuracy	Precision	Recall	F1-Score
SGD -LR: 0.01 -Mom: 0.01	50	0.3640	80.6%	66.3%	95.3%	78.2%
Adam -LR: 0.01	44	0.3962	81.9%	69.5%	90.0%	78.4%
Adam -LR: 0.001	50	0.3131	85.0%	72.4%	95.7%	82.4%
Adam -LR: 0.0001	44	0.3434	83.2%	72.4%	87.5%	79.2%

C. BERT METRICS

Table 13 presents the results of the best-performing neural network trained with BERT embeddings and trainable embedding model layers, while Table 14 provides the results with non-trainable layers. Unfreezing embedding model layers differs between word2Vec and BERT models. Word2vec models use a single hidden layer to learn word-level semantics, whereas BERT was built with 12 hidden layers designed for a deeper context-aware understanding of language.

The best-performing LSTM model using BERT embeddings was built with unfrozen embedding layers and trained using the SGD optimizer, configured with a learning rate of 0.0001 and a momentum of 0.001 (Fig. 10). With unfrozen embedding layers, LSTMs trained with BERT embeddings and the SGD optimizer achieved a maximum accuracy range of 87–88%, with the top model reaching an accuracy of 88.8%.

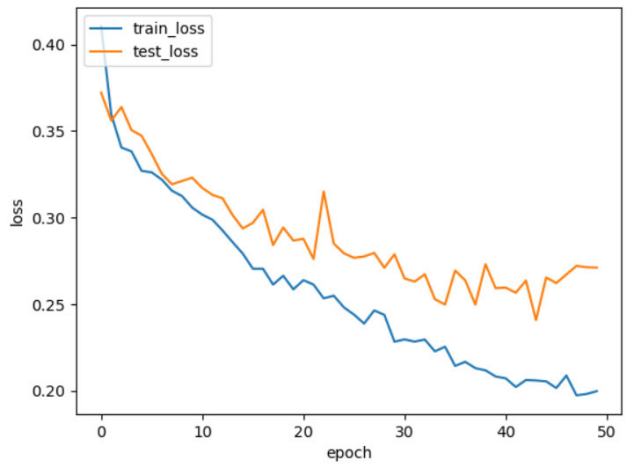


FIGURE 9. Loss graph of LSTM using CBOW embeddings with Adam optimizer and learning rate 0.001(Unfrozen Embeddings).

TABLE 11. word2vec: CBOW metrics (Unfrozen Layers).

Hyper-parameters	Epoch	Loss	Accuracy	Precision	Recall	F1-Score
SGD -LR: 0.01 -Mom: 0.01	43	0.3417	83.0%	71.8%	88.5%	79.3%
SGD -LR: 0.0001 -Mom: 0.01	50	0.5920	69.8%	67.9%	33.3%	44.7%
SGD -LR: 0.0001 -Mom: 0.001	42	0.6246	69.0%	64.2%	34.8%	45.1%
SGD -LR: 0.0001 -Mom: 0.0001	37	0.6159	68.7%	63.4%	34.8%	44.9%
Adam -LR: 0.01	46	0.3531	83.4%	70.7%	93.5%	80.6%
Adam -LR: 0.001	34	0.2528	87.5%	78.0%	91.8%	84.3%
Adam -LR: 0.0001	45	0.2990	86.5%	77.0%	90.0%	83.0%

TABLE 12. word2vec: CBOW metrics (Frozen Layers).

Hyper-parameters	Epoch	Loss	Accuracy	Precision	Recall	F1-Score
SGD -LR: 0.01 -Mom: 0.01	45	0.3419	83.3%	72.5%	87.8%	79.4%
SGD -LR: 0.0001 -Mom: 0.01	33	0.5996	69.6%	69.4%	30.8%	43.7%
Adam -LR: 0.01	25	0.3709	82.0%	68.5%	94.3%	79.3%
Adam -LR: 0.001	35	0.2710	87.5%	79.5%	88.9%	83.9%
Adam -LR: 0.0001	50	0.2654	86.9%	77.9%	89.6%	83.3%

In contrast to word2vec models, those trained with the Adam optimizer struggled to achieve significant accuracy gains. The graphs (Fig. 11) of these models indicate that additional epochs would not eventually result in higher-performance models.

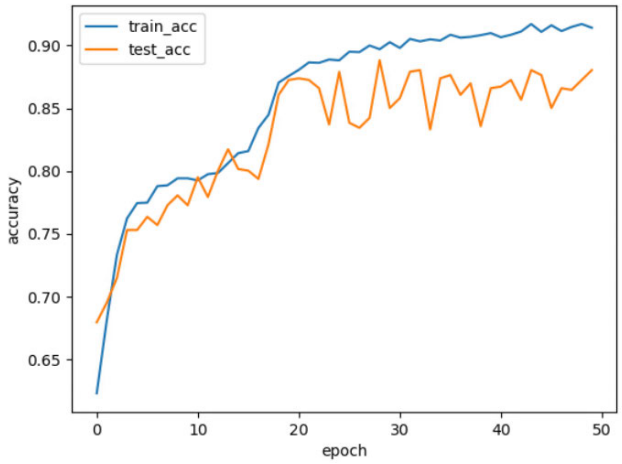


FIGURE 10. Accuracy graph of LSTM using BERT embeddings with SGD optimizer, learning rate 0.0001 and momentum 0.001 (Unfrozen Embeddings).

TABLE 13. BERT metrics (Unfrozen Layers).

Hyper-parameters	Epoch	Loss	Accuracy	Precision	Recall	F1-Score
SGD -LR: 0.01 -Mom: 0.01	19	0.2862	87.5%	81.7%	84.9%	83.3%
SGD -LR: 0.0001 -Mom: 0.01	20	0.2913	88.6%	81.0%	90.0%	85.2%
SGD -LR: 0.0001 -Mom: 0.001	29	0.2625	88.8%	80.1%	92.5%	85.9%
SGD -LR: 0.0001 -Mom: 0.0001	47	0.2909	87.3%	83.0%	82.1%	82.5%

D. ROBERTA METRICS

Tables 15 (Unfrozen Layers) and 16 (Frozen Layers) present the results of the best-performing models trained with RoBERTa embeddings. Similarly to models built with BERT embeddings, the LSTM model with the highest performance achieved an accuracy of 88.8% and an F1-score of 84.2%.

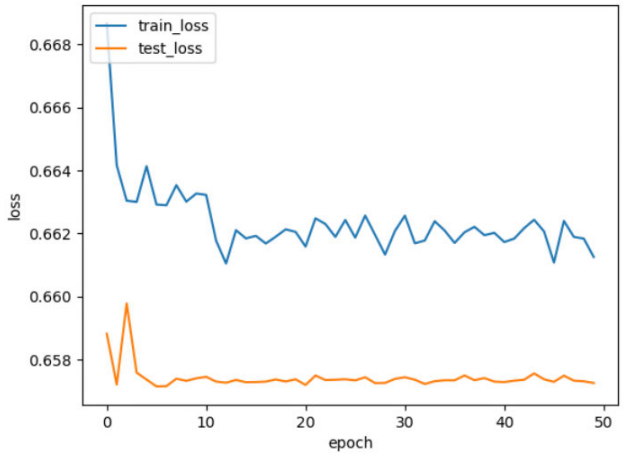


FIGURE 11. Loss graph of LSTM using BERT embeddings with Adam optimizer and learning rate 0.001 (Unfrozen Embeddings).

TABLE 14. BERT metrics (Frozen Layers).

Hyper-parameters	Epoch	Loss	Accuracy	Precision	Recall	F1-Score
SGD -LR: 0.01 -Mom: 0.01	50	0.3820	82.8%	78.5%	73.1%	75.7%
SGD -LR: 0.0001 -Mom: 0.01	48	0.4999	79.9%	76.5%	65.2%	70.4%
SGD -LR: 0.0001 -Mom: 0.001	49	0.4603	81.3%	77.7%	68.8%	73.0%
SGD -LR: 0.0001 -Mom: 0.0001	38	0.4838	79.4%	78.0%	60.9%	68.4%
Adam -LR: 0.001	49	0.4293	82.9%	69.8%	94.3%	80.2%
Adam -LR: 0.0001	34	0.3687	84.8%	78.0%	81.4%	79.6%

This was achieved using an embedding model with trainable layers and the SGD optimizer configured with a learning rate of 0.0001 and a momentum of 0.001. Fig. 12 provides the accuracy graph of this model; as shown, the model struggled to achieve stability until around epoch 20. At this point, the model stabilizes and achieves consistent performance.

For models trained with RoBERTa embeddings and unfrozen layers, the SGD optimizer consistently achieved accuracies of approximately 88%. However, as with models trained using BERT embeddings, those trained with the Adam optimizer failed to achieve significant accuracy gains.

TABLE 15. RoBERTa metrics (Unfrozen Layers).

Hyper-parameters	Epoch	Loss	Accuracy	Precision	Recall	F1-Score
SGD -LR: 0.01 -Mom: 0.01	31	0.2625	86.6%	82.2%	81.0%	81.6%
SGD -LR: 0.0001 -Mom: 0.01	47	0.2454	88.7%	79.0%	94.3%	85.9%
SGD -LR: 0.0001 -Mom: 0.001	45	0.2708	88.8%	87.6%	81.0%	84.2%
SGD -LR: 0.0001 -Mom: 0.0001	49	0.2868	88.6%	84.0%	84.9%	84.5%

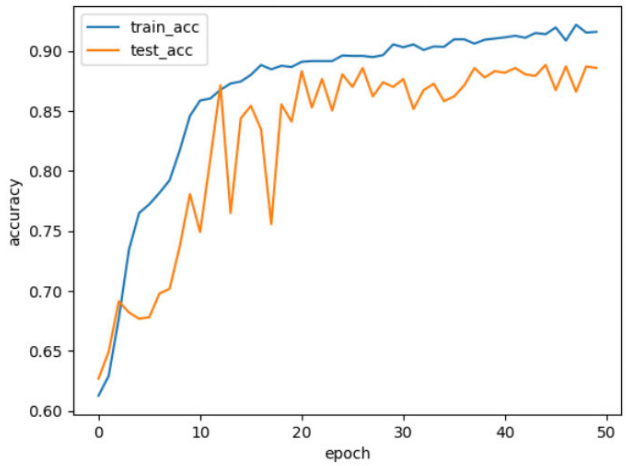


FIGURE 12. Accuracy graph of LSTM using RoBERTa embeddings with SGD optimizer, learning rate 0.0001 and momentum 0.001 (Unfrozen Embeddings).

TABLE 16. RoBERTa metrics (Frozen Layers).

Hyper-parameters	Epoch	Loss	Accuracy	Precision	Recall	F1-Score
SGD -LR: 0.01 -Mom: 0.01	46	0.3032	85.3%	76.0%	87.5%	81.3%
SGD -LR: 0.0001 -Mom: 0.01	48	0.4126	82.7%	79.3%	71.3%	75.1%
SGD -LR: 0.0001 -Mom: 0.001	49	0.4346	81.5%	77.2%	70.3%	73.5%
SGD -LR: 0.0001 -Mom: 0.0001	46	0.4742	80.0%	77.5%	64.2%	70.2%
Adam -LR: 0.001	34	0.3275	85.9%	76.9%	88.2%	82.1%
Adam -LR: 0.0001	44	0.3805	84.8%	80.8%	76.7%	78.7%

E. GPT-1 RESULTS

As demonstrated in Tables 17 and 18, the results of the LSTMs trained using GPT-1 embeddings performed poorly compared to LSTMs trained using embeddings from other models. The accuracy (Fig. 13) and loss (Fig. 14) graphs for the best-performing LSTM demonstrate that the model began to destabilize around epoch 15, and the loss graph indicates the emergence of overfitting. The LSTM with the highest accuracy (79.5%) and F1-score (68.7%) was achieved using the SGD optimizer with a learning rate and momentum of 0.01, along with trainable embedding layers.

F. GPT-2 RESULTS

Table 19 presents the results of LSTM training using GPT-2 embeddings with unfrozen layers. Two-thirds of the models trained using the Adam optimizer failed to surpass the baseline accuracy of assigning all samples to the same category. In contrast, models trained with the SGD optimizer achieved significantly better performance. The top-performing model, trained with the SGD optimizer a learning rate of 0.0001 and a momentum of 0.01, achieved an accuracy of 92.5% and an F1-score of 89.7%. As shown in Fig. 15, this model required a few epochs to stabilize but reaches a high degree of stability around epoch 20.

Table 20 summarizes the results of training LSTMs using embeddings from a GPT-2 model with frozen hidden layers. The best-performing neural network in this configuration used the Adam optimizer with a learning rate of 0.001, and achieved an accuracy of 87.9% and an F1-score of 84.5%.

TABLE 17. GPT metrics (Unfrozen Layers).

Hyper-parameters	Epoch	Loss	Accuracy	Precision	Recall	F1-Score
SGD -LR: 0.01 -Mom: 0.01	27	0.4366	79.5%	78.1%	61.3%	68.7%
SGD -LR: 0.0001 -Mom: 0.01	41	0.4328	77.9%	77.1%	56.6%	65.3%
SGD -LR: 0.0001 -Mom: 0.001	45	0.3949	78.8%	77.3%	59.9%	67.5%
SGD -LR: 0.0001 -Mom: 0.0001	48	0.3958	78.2%	80.2%	53.8%	64.4%

TABLE 18. GPT metrics (Frozen Layers).

Hyper-parameters	Epoch	Loss	Accuracy	Precision	Recall	F1-Score
SGD -LR: 0.01 -Mom: 0.01	20	0.4955	74.8%	75.1%	46.6%	57.5%
SGD -LR: 0.0001 -Mom: 0.01	43	0.4778	74.1%	68.0%	55.6%	61.1%
SGD -LR: 0.0001 -Mom: 0.001	46	0.4672	74.8%	71.2%	52.3%	60.3%
SGD -LR: 0.0001 -Mom: 0.0001	39	0.4738	75.2%	69.2%	58.1%	63.2%
Adam -LR: 0.001	16	0.5036	74.1%	68.1%	55.2%	61.0%
Adam -LR: 0.0001	4	0.4654	75.4%	73.5%	51.6%	60.6%

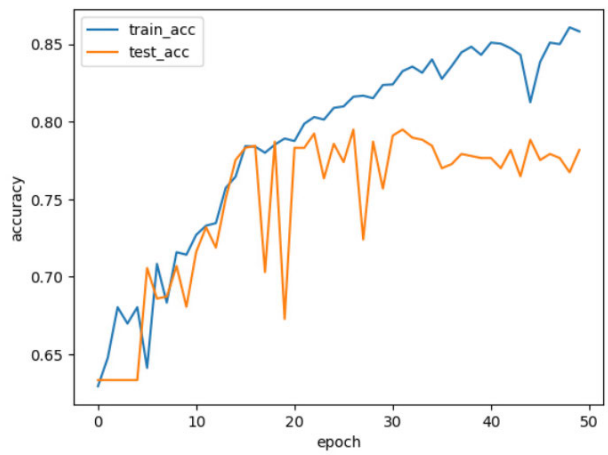


FIGURE 13. Accuracy graph of LSTM using GPT-1 embeddings with SGD optimizer, learning rate 0.01 and momentum 0.01 (Unfrozen Embeddings).

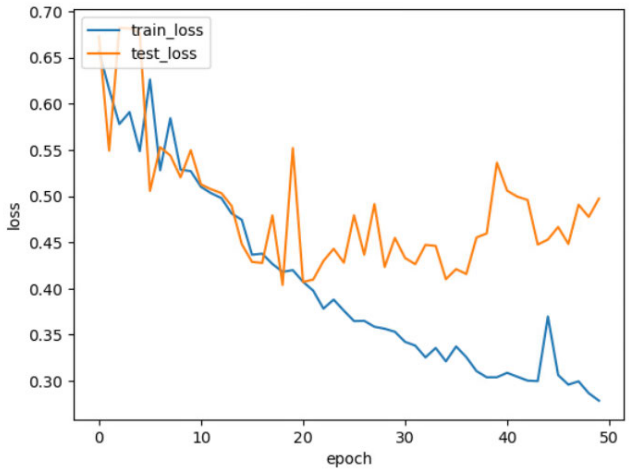


FIGURE 14. Loss graph of LSTM Using GPT-1 embeddings with SGD optimizer, learning rate 0.01 and momentum 0.01 (Unfrozen Embeddings).

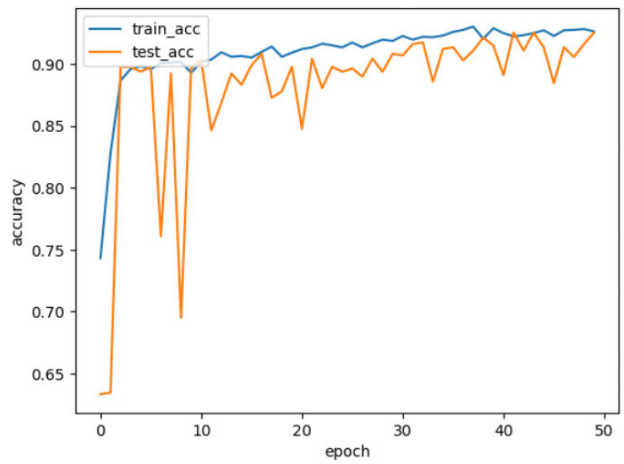


FIGURE 15. Accuracy graph of LSTM using GPT-2 embeddings with SGD optimizer, learning rate 0.0001 and momentum 0.01 (Unfrozen Embeddings).

TABLE 19. GPT-2 metrics (Unfrozen Layers).

Hyper-parameters	Epoch	Loss	Accuracy	Precision	Recall	F1-Score
SGD -LR: 0.01 -Mom: 0.01	50	0.3049	88.6%	84.5%	84.2%	84.4%
SGD -LR: 0.0001 -Mom: 0.01	44	0.1588	92.5%	90.2%	89.2%	89.7%
SGD -LR: 0.0001 -Mom: 0.001	42	0.1712	92.1%	91.6%	86.4%	88.9%
SGD -LR: 0.0001 -Mom: 0.0001	46	0.1522	92.1%	91.3%	86.7%	89.0%
Adam -LR: 0.0001	17	0.3996	87.1%	79.1%	88.2%	83.4%

G. COMPARISON OF GPT-1, GPT-2, BERT, ROBERTA, CBOW, AND SKIP-GRAM

Table 21 compares the best-performing models created using embeddings from each embedding model. Although GPT-2 models achieved slightly higher performance than those built with Skip-Gram embeddings, the simpler Skip-Gram models performed remarkably well, particularly considering they use on a single hidden layer, compared to the 12 hidden layers utilized by GPT-2 embedding models.

TABLE 20. GPT-2 metrics (Frozen LLM Layers).

Hyper-parameters	Epoch	Loss	Accuracy	Precision	Recall	F1-Score
SGD -LR: 0.01 -Mom: 0.01	15	0.3862	82.9%	77.9%	74.6%	76.2%
SGD -LR: 0.0001 -Mom: 0.01	47	0.3210	86.9%	76.6%	92.5%	83.8%
SGD -LR: 0.0001 -Mom: 0.001	42	0.3288	85.9%	76.1%	90.0%	82.4%
SGD -LR: 0.0001 -Mom: 0.0001	33	0.3615	82.8%	81.4%	68.8%	74.6%
Adam -LR: 0.001	34	0.2712	87.9%	79.7%	90.0%	84.5%
Adam -LR: 0.0001	40	0.2605	87.8%	77.4%	94.3%	85.0%

TABLE 21. Best performing neural networks.

	Unfrozen Layers			Frozen Layers		
Embeddings	Hyper Parameters	Accuracy	F1-Score	Hyper Parameters	Accuracy	F1-Score
Skip-Gram	Adam -LR: 0.001	92.0%	89.6%	Adam -LR: 0.001	85.0%	82.4%
CBOW	Adam -LR: 0.001	87.5%	84.3%	Adam -LR: 0.001	87.5%	83.9%
BERT	SGD -LR: 0.0001 -Mom: 0.001	88.8%	85.9%	Adam -LR: 0.0001	84.8%	79.6%
RoBERTa	SGD -LR: 0.0001 -Mom: 0.001	88.8%	84.2%	Adam -LR: 0.001	85.9%	82.1%
GPT-1	SGD -LR: 0.01 -Mom: 0.01	79.5%	68.7%	Adam -LR: 0.0001	75.4%	60.6%
GPT-2	SGD -LR: 0.0001 -Mom: 0.01	92.5%	89.7%	Adam -LR: 0.001	87.9%	84.5%

VI. DISCUSSION

A. GPT-2 VS. BERT & ROBERTA EMBEDDINGS

The neural networks trained with GPT-2 embeddings significantly outperformed their bidirectional counterparts (BERT and RoBERTa). These findings suggest that embeddings produced by unidirectional models, which simulate the sequential manner in which humans typically read text, were more effective at capturing semantics in lifted code compared to bidirectional models. Interestingly, [48] highlights that BERT excels at classification tasks, suggesting it might have performed well in the code classification tasks in this study. However, it is important to note that BERT was not directly used for classification in this implementation. Instead, BERT provided token-level embeddings to an LSTM neural network, which was ultimately responsible for classifying vulnerable code samples. In contrast, as demonstrated in [50], GPT models excel at generative tasks. This suggests that the generative capabilities of GPT models may better capture the code-level semantics required by neural networks for

classification tasks than bidirectional models. Nevertheless, additional testing is necessary to validate this potential conclusion.

B. GPT-2 VS. SKIP-GRAM EMBEDDINGS

The best-performing LSTM model built with GPT-2 embeddings slightly outperformed the best model built with Skip-Gram embeddings. The GPT-2 model surpassed the Skip-Gram model by 0.5% in accuracy and 0.1% in F1-score. However, the Skip-Gram model achieved a recall that was 5.4% higher than the best model trained with GPT-2 embeddings. At first glance, these results suggest that both models produce comparable performance. Nevertheless, it is noteworthy that models built using the SGD optimizer and a learning rate of 0.0001 consistently achieved accuracy scores above 92%. In contrast, models built with Skip-Gram embeddings, aside from the best-performing neural network, consistently produced top scores in the mid-80% range. Thus, not only did the best model built with GPT-2 embeddings achieve higher accuracy, precision, and F1-scores than its Skip-Gram counterpart, but it also maintained this high accuracy even when the momentum hyperparameter was adjusted. These findings indicate that GPT-2 embeddings are better suited for the task outlined in this study.

C. BERT, ROBERTA, & GPT-1 VS. SKIP-GRAM EMBEDDINGS

The best-performing neural networks trained using Skip-Gram embeddings outperformed those utilizing BERT, RoBERTa, or GPT-1 embeddings. This suggests that, with smaller datasets, the simpler techniques employed by Skip-Gram models may be more effective at capturing code-level semantics than the more complex transformer-based models. Transformer models, however, are typically trained on much larger datasets than the modest dataset used in this project. For example, the original BERT model was trained on BooksCorpus (800 million words) and English Wikipedia (2.5 billion words) [48], while RoBERTa was trained on datasets ten times larger [49]. Despite using a modest dataset of approximately 48,000 samples, the LSTMs trained with embeddings from the BERT and RoBERTa models achieved an impressive 88.8% accuracy.

D. SKIP-GRAM VS. CBOW EMBEDDINGS

The neural networks built using Skip-Gram embeddings outperformed those using CBOW embeddings. Skip-Gram models are particularly effective at learning token-level embeddings when the sample size is limited [47]. In contrast, CBOW models excel at generating embeddings for high-frequency tokens, which aligns with this dataset, as the word2vec model reported only 332 unique tokens at the end of training [47]. Despite the high-token frequency, which typically favors CBOW, the Skip-Gram embeddings likely held an advantage due to the smaller training dataset.

VII. FUTURE WORK

A. DATASET SIZES

The datasets used to train the transformer-based models in this study were relatively modest compared to the massive datasets typically used for such models. Despite the limited number of training samples, the LSTMs built with these embeddings performed reasonably well. The models built with GPT-2 embeddings consistently achieved over 92% accuracy when trained with the SGD optimizer and a learning rate of 0.0001. However, performance could potentially improve further with larger datasets for training the embedding models.

B. EVALUATE OTHER EMBEDDING MODELS

As mentioned in Section III-E, there are several transformer-based embedding models beyond those evaluated in this implementation of TEDVIL. Many of these models incorporate architectural choices that could substantially impact how embeddings are generated. Future research should explore and compare these models in the context of TEDVIL to determine their effects on the generated embeddings.

C. NEURAL NETWORKS AND HYPERPARAMETERS

LSTM neural networks were chosen for their particular implementation of TEDVIL. However, there are several other neural networks such as SRNN, BLSTM, GRU, BGRU, CNNs, etc. It may be possible to achieve higher accuracy with a different neural network. Furthermore, neural networks such as GNNs can be used to model program flow, potentially improving model performance.

D. LLVM FUNCTION SIZE

In this study, samples containing more than 2,048 tokens were excluded from the dataset used to train neural networks for vulnerability detection due to memory limitations. As a result, nearly 100 samples were removed from this dataset. Future research should explore methods for handling samples with long token lengths without overextending system resources.

E. DATA SNOOPING

Reference [68] highlights that one of the most prevalent design flaws in applying machine learning to cybersecurity problems is data snooping. A specific type of data snooping, known as test snooping, occurs when there is insufficient separation between datasets used to train and validate machine learning models; this flaw can manifest when data used to train embedding models is reused to validate performance, leading to potentially inflating evaluation metrics [68]. Although the research described in this paper avoids this issue, [69] explored the effects of data snooping on the Skip-Gram, CBOW, RoBERTa, and GPT-2 models. Their findings indicate that even when a test snooping design flaw was introduced, the ranking of the best-performing models remained consistent (GPT-2, Skip-Gram, RoBERTa,

and CBOW). Moreover, the accuracy differences between the best-performing models in each study were minimal: GPT-2 differed by only 0.4%, Skip-Gram by 1.5%, RoBERTa by 0.3%, and CBOW by 0.4%.

F. EMBEDDING LAYER TUNING ENHANCEMENTS

This study employed a binary approach to freezing or unfreezing embedding model layers. However, [70] emphasizes the advantages of methods such as discriminative fine-tuning and gradual unfreezing. If implemented, these approaches could lead to potential improvements in model performance.

G. VULNERABILITY COVERAGE

This study's implementation of TEDVIL focused on identifying CWE-121 vulnerabilities. However, other classifications of vulnerabilities exist, including use-after-free, null pointer dereference, and heap corruption vulnerabilities. Future research should explore the application of TEDVIL to these and other vulnerability types.

VIII. RELATED WORK

Several studies have utilized transformer-based embedding models to detect vulnerabilities in source code. Examples include BERT [71], [72], [73], [74], [75], [76], [77], RoBERTa [74], [77], [78], GPT-based models [79], [80], [81], [82], [83], [84], [85], [86], [87], [88], [89], and DistilBERT [74], [77], [90], [91]. Furthermore, these models have been utilized for code similarity tasks, encompassing both bidirectional [33], [34], [35], [36] and unidirectional [37] approaches.

The implementation of TEDVIL in this study utilized C/C++ samples from the NIST SARD [53] Juliet 1.3 source code dataset. Datasets from this project have been extensively used in research focused on identifying vulnerabilities in compiled code. References [17], [18], [19], and [26] mentions using the SARD dataset for their research but do not specify the exact dataset employed. In contrast, [20], [23], [27], [28], [30], [31], [92], [93] explicitly state that they used the Juliet dataset. Additionally, [94] reports using MIT Lincoln Laboratory's MLL05b dataset, and [30] utilized the STONESOUP datasets (IARPA12 and IARPA14), all of which are available in the SARD project.

Several projects have focused on identifying vulnerabilities in compiled code and have employed the word2vec model to generate token-level embeddings [15], [18], [19], [22], [23], [25], [29]. However, there are known limitations associated with using word2vec in this capacity. To address these weaknesses, several projects have proposed their own solutions, which are either inspired by word2vec or incorporate it as part of their approach. These models include Instruction2vec [30], Bin2vec [20], VDGraph2Vec [31], BinVulDet [28], and VulANalyzeR [27].

There have been relatively few projects that utilized transformer-based models to detect vulnerabilities in compiled code. For example, [31] trained a RoBERTa model on assembly instructions and leveraged these embeddings to train an MPNN, enabling it to distinguish between vulnerable and non-vulnerable code. Similarly, [38] trained a RoBERTa model using LLVM IR and employed these embeddings to train a neural network to identify vulnerabilities in IR code. Finally, [39] trained a BERT model with P-Code and used these embeddings to detect vulnerabilities in P-Code samples.

Several studies have employed recurrent neural networks (RNNs) to detect vulnerabilities in compiled code. For example, [15], [16], [17], [18], [19], [20], [21], [22], [23] utilized long short-term memory networks (LSTMs) for this purpose, while [17], [18], [19], [21], [23], [25], [26], [27] adopted the bidirectional variant of LSTMs (BLSTMs). Furthermore, [18], [19], [23], [24] employed gated recurrent units (GRUs), and [17], [18], [19], [21], [23], [26], [27], [28] used the bidirectional variant of GRUs (BGRUs) for a similar purpose.

The aspects of TEDVIL, as well as the specific implementation adopted by the research for dataset selection, lifting assembly code to LLVM, and preprocessing the lifted code in this study, are strongly influenced by the approaches described in [18] and [23]. These studies leveraged the SARD source code dataset, compiling the samples, and converting the resulting compiled code to LLVM IR using the RetDec tool. The lifted LLVM code was then employed to train a word2vec model, which generated embeddings subsequently used to train a neural network for vulnerability detection. Building on these foundations, the current research extended this approach by utilizing LLVM IR to train custom transformer-based embedding models (i.e., BERT, RoBERTa, GPT-1, and GPT-2). These embeddings were further used to train multiple LSTM neural networks with varying hyperparameters, enabling a comparative analysis of LSTMs trained using multiple embedding models.

IX. CONCLUSION

This study introduced TEDVIL, a novel framework that leverages transformer-based embeddings to detect vulnerabilities in lifted code. Using this approach, LSTMs trained with GPT-2 embeddings achieved strong performance, attaining an accuracy of 92.5% and an F1-score of 89.7%. These models outperformed LSTMs trained with embeddings from bidirectional transformers (e.g., BERT and RoBERTa) and simpler word2Vec models (e.g., Skip-Gram and CBOW). These findings underscore the potential of transformer-based models for vulnerability detection in compiled code, even in resource-constrained settings.

This study highlights the influence of optimizers on the performance of neural networks trained with transformer-based embeddings and the simpler word2vec models. In all cases, neural networks trained with embedding models that had trainable layers outperformed those with frozen layers. However, networks using word2Vec embeddings achieved optimal performance with the Adam optimizer, whereas LSTMs trained with transformer-based embeddings performed better with the SGD optimizer. LSTMs built with

GPT-2 embeddings consistently outperformed those built with other embeddings, underscoring their suitability for vulnerability detection tasks.

The promising results of this study open multiple avenues for future exploration. Future implementations of TEDVIL could evaluate additional transformer architectures (e.g., XLNet, Longformer) or neural network models (e.g., BLSTM, GRU). Furthermore, the framework's performance with larger datasets or alternative lifted code representations, such as VEX, warrants further investigation.

ACKNOWLEDGMENT

Grammarly, ChatGPT, and Writeful assisted with the grammar, wording, and spelling of this work.

REFERENCES

[1] N. P. Essien and M. C. Umo, "Conceptual analysis of optimization strategies in operating system (windows and unix) in the current it trend from 2015 to 2023," *Int. J. Contemp. Afr. Res. Netw.*, vol. 2, no. 1, pp. 1–15, Jun. 2024.

[2] S. Hussain, F. Bahadur, F. Gul, A. Iqbal, and S. Nazeer, "Comparative study of open source verses closed source with respect to user's perspectives," *Int. J. Comput.*, vol. 17, no. 1, pp. 30–34, Aug. 2015. [Online]. Available: https://www.ijcjournal.org/index.php/InternationalJournalOfComputer/article/view/412

[3] M. Zineddine, C. Alaoui, and N. Saidou, "Commercial software companies and open source community reaction to disclosed vulnerabilities: Case of windows server 2008 and Linux patching," in *Proc. Int. Conf. Wireless Technol., Embedded Intell. Syst. (WITS)*, Apr. 2017, pp. 1–4, doi: 10.1109/WITS.2017.7934677.

[4] I. Haller, A. Slowinska, and H. Bos, "Scalable data structure detection and classification for C/C++ binaries," *Empirical Softw. Eng.*, vol. 21, no. 3, pp. 778–810, Jun. 2016, doi: 10.1007/s10664-015-9363-y.

[5] A. Ballman and D. Svoboda, "Avoiding insecure C++—How to avoid common C++ security vulnerabilities," in *Proc. IEEE Cybersecurity Develop. (SecDev)*, Nov. 2016, p. 65, doi: 10.1109/SECDEV.2016.022.

[6] S. B. Wicker, "The ethics of zero-day exploits—The NSA meets the trolley car," *Commun. ACM*, vol. 64, no. 1, pp. 97–103, Jan. 2021, doi: 10.1145/3393670.

[7] X. Jiang and D. Xu, "Profiling self-propagating worms via behavioral footprinting," in *Proc. 4th ACM Workshop Recurring Malcode*, Nov. 2006, pp. 17–24, doi: 10.1145/1179542.1179547.

[8] P. Porras, H. Saidi, and V. Yegneswaran, "An analysis of conficker's logic and rendezvous points," Dept. Computer Science Laboratory, SRI Int., Menlo Park, CA, USA, Tech. Rep., 2009. [Online]. Available: https://www.csl.sri.com/users/vinod/papers/Conficker/

[9] H. Berghel, "The code red worm," *Commun. ACM*, vol. 44, no. 12, pp. 15–19, Dec. 2001, doi: 10.1145/501317.501328.

[10] H. Oz, A. Aris, A. Levi, and A. S. Uluagac, "A survey on ransomware: Evolution, taxonomy, and defense solutions," *ACM Comput. Surv.*, vol. 54, no. 11s, pp. 1–37, Sep. 2022, doi: 10.1145/3514229.

[11] O. Kovalchuk, M. Shynkaryk, and M. Masonkova, "Econometric models for estimating the financial effect of cybercrimes," in *Proc. 11th Int. Conf. Adv. Comput. Inf. Technol. (ACIT)*, Sep. 2021, pp. 381–384, doi: 10.1109/ACIT52158.2021.9548490.

[12] N. Kshetri and J. Voas, "Ransomware as a business (RaaB)," *IT Prof.*, vol. 24, no. 2, pp. 83–87, Mar. 2022, doi: 10.1109/MITP.2022.3157208.

[13] C. S. Anand and R. Shanker, "Advancing crypto ransomware with multi level extortion: A peril to critical infrastructure," in *Proc. 2nd Int. Conf. Innov. Technol. (INOCON)*, Mar. 2023, pp. 1–5, doi: 10.1109/INOCON57975.2023.10100971.

[14] C. McGlave, H. Neprash, and S. Nikpay, "Hacked to pieces? The effects of ransomware attacks on hospitals and patients," SSRN Scholarly Paper, 2023. [Online]. Available: https://doi.org/10.2139/ssrn.4579292

[15] A. Pechenkin and R. Demidov, "Application of deep neural networks for security analysis of digital infrastructure components," in *Proc. SHS Web Conf.*, vol. 44, Jan. 2018, p. 00068, doi: 10.1051/shsconf/20184400068.

[16] W. Arild Dahl, L. Erdodi, and F. Massimo Zennaro, "Stack-based buffer overflow detection using recurrent neural networks," 2020, *arXiv:2012.15116*.

[17] A. Aumpansub and Z. Huang, "Learning-based vulnerability detection in binary code," in *Proc. 14th Int. Conf. Mach. Learn. Comput. (ICMLC)*, Feb. 2022, pp. 266–271, doi: 10.1145/3529836.3529926.

[18] J. Zheng, J. Pang, X. Zhang, X. Zhou, M. Li, and J. Wang, "Recurrent neural network based binary code vulnerability detection," in *Proc. 2nd Int. Conf. Algorithms, Comput. Artif. Intell.*, Dec. 2019, pp. 160–165, doi: 10.1145/3377713.3377738.

[19] J. Tian, W. Xing, and Z. Li, "BVDetector: A program slice-based binary code vulnerability intelligent detection system," *Inf. Softw. Technol.*, vol. 123, Jul. 2020, Art. no. 106289, doi: 10.1016/j.infsof.2020.106289.

[20] S. Arakelyan, S. Arasteh, C. Hauser, E. Kline, and A. Galstyan, "Bin2vec: Learning representations of binary executable programs for security tasks," *Cybersecurity*, vol. 4, no. 1, p. 26, Dec. 2021, doi: 10.1186/s42400-021-00088-4.

[21] A. Aumpansub and Z. Huang, "Detecting software vulnerabilities using neural networks," in *Proc. 13th Int. Conf. Mach. Learn. Comput.*, Feb. 2021, pp. 166–171, doi: 10.1145/3457682.3457707.

[22] H. Liang, Z. Xie, Y. Chen, H. Ning, and J. Wang, "FIT: Inspect vulnerabilities in cross-architecture firmware by deep learning and bipartite matching," *Comput. Secur.*, vol. 99, Dec. 2020, Art. no. 102032, doi: 10.1016/j.cose.2020.102032.

[23] A. Schaad and D. Binder, "Deep-Learning-based vulnerability detection in binary executables," in *Proc. Int. Symp. Found. Pract. Secur.*, Jan. 2022, pp. 453–460, doi: 10.48550/arxiv.2212.01254.

[24] T. Chappelly, C. Cifuentes, P. Krishnan, and S. Gevay, "Machine learning for finding bugs: An initial report," in *Proc. IEEE Workshop Mach. Learn. Techn. Softw. Quality Eval. (MaLTeSQuE)*, Feb. 2017, pp. 21–26, doi: 10.1109/MALTESQUE.2017.7882012.

[25] A. Pechenkin and R. Demidov, "Applying deep learning and vector representation for software vulnerabilities detection," in *Proc. 11th Int. Conf. Secur. Inf. Netw.*, Sep. 2018, pp. 1–6, doi: 10.1145/3264437.3264489.

[26] Z. Li, D. Zou, S. Xu, Z. Chen, Y. Zhu, and H. Jin, "VulDeeLocator: A deep learning-based fine-grained vulnerability detector," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2821–2837, Jul. 2022, doi: 10.1109/TDSC.2021.3076142.

[27] L. Li, S. H. H. Ding, Y. Tian, B. C. M. Fung, P. Charland, W. Ou, L. Song, and C. Chen, "VulANalyzeR: Explainable binary vulnerability detection with multi-task learning and attentional graph convolution," *ACM Trans. Privacy Secur.*, vol. 26, no. 3, pp. 1–25, Aug. 2023, doi: 10.1145/3585386.

[28] Y. Wang, P. Jia, X. Peng, C. Huang, and J. Liu, "BinVulDet: Detecting vulnerability in binary program via decompiled pseudo code and BiLSTM-attention," *Comput. Secur.*, vol. 125, Feb. 2023, Art. no. 103023, doi: 10.1016/j.cose.2022.103023.

[29] G. Grieco, G. L. Grinblat, L. Uzal, S. Rawat, J. Feist, and L. Mounier, "Toward large-scale vulnerability discovery using machine learning," in *Proc. 6th ACM Conf. Data Appl. Secur. Privacy*, Mar. 2016, pp. 85–96, doi: 10.1145/2857705.2857720.

[30] Y. Lee, H. Kwon, S.-H. Choi, S.-H. Lim, S. H. Baek, and K.-W. Park, "instruction2vec: Efficient preprocessor of assembly code to detect software weakness with CNN," *Appl. Sci.*, vol. 9, no. 19, p. 4086, Sep. 2019, doi: 10.3390/app9194086.

[31] A. Diwan, M. Q. Li, and B. C. M. Fung, "VDGraph2 Vec: Vulnerability detection in assembly code using message passing neural networks," in *Proc. 21st IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Dec. 2022, pp. 1039–1046, doi: 10.1109/ICMLA55696.2022.00173.

[32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst. (NIPS)*, vol. 30, Red Hook, NY, USA, Jun. 2017, pp. 5998–6008, doi: 10.5555/3295222.3295349.

[33] P. Shirani, S. Bhatt, A. Hailane, and G.-V. Jourdan, "Towards cross-architecture binary code vulnerability detection," in *Proc. 33rd Annu. Int. Conf. Comput. Sci. Softw. Eng. (CASCON)*, 2023, pp. 191–196, doi: 10.5555/3615924.3615947.

[34] H. Koo, S. Park, D. Choi, and T. Kim, "Semantic-aware binary code representation with BERT," 2021, *arXiv:2106.05478*.

[35] Y. Yu, S. Gan, and X. Qin, "FirmVulSeeker—BERT and Siamese network-based vulnerability search for embedded device firmware images," *J. Internet Things*, vol. 4, no. 1, pp. 1–20, 2022, doi: 10.32604/jiot.2022.019469.

[36] S. Ahn, S. Ahn, H. Koo, and Y. Paek, "Practical binary code similarity detection with BERT-based transferable similarity learning," in *Proc. 38th Annu. Comput. Secur. Appl. Conf.*, Dec. 2022, pp. 361–374, doi: 10.1145/3564625.3567975.

[37] X. Li, Y. Qu, and H. Yin, "PalmTree: Learning an assembly language model for instruction embedding," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3236–3251, doi: 10.1145/3460120.3484587.
[38] S. K. Gallagher, W. E. Klieber, and D. Svoboda, "LLVM intermediate representation for code weakness identification," Defense Tech. Inf. Center, Fort Belvoir, VA, USA, Tech. Rep. AD1178536, 2022.
[39] W. Han, J. Pang, X. Zhou, and D. Zhu, "Binary vulnerability mining technology based on neural network feature fusion," in *Proc. 5th Int. Conf. Adv. Electron. Mater., Comput. Softw. Eng. (AEMCSE)*, Apr. 2022, pp. 257–261, doi: 10.1109/AEMCSE55572.2022.00058.
[40] G. A. McCully, J. D. Hastings, S. Xu, and A. Fortier, "Bi-directional transformers vs. Word2vec: Discovering vulnerabilities in lifted compiled code," in *Proc. Cyber Awareness Res. Symp. (CARS)*, Oct. 2024, pp. 1–8, doi: 10.1109/cars61786.2024.10778724.
[41] G. A. McCully, J. D. Hastings, S. Xu, and A. Fortier, "Comparing unidirectional, bidirectional, and Word2vec models for discovering vulnerabilities in compiled lifted code," 2024, *arXiv:2409.17513*.
[42] LLVM Developer Group. *LLVM Language Reference Manual*. Accessed: Dec. 10, 2024. [Online]. Available: https://llvm.org/docs/LangRef.html
[43] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *Proc. Int. Symp. Code Gener. Optim.*, Mar. 2004, pp. 75–86, doi: 10.1109/CGO.2004.1281665.
[44] B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Global value numbers and redundant computations," in *Proc. 15th ACM SIGPLAN-SIGACT Symp. Princ. Program. Lang.*, 1988, pp. 12–27, doi: 10.1145/73560.73562.
[45] A. Harper, R. Linn, S. Sims, M. Baucom, H. Tejeda, D. Fernandez, and M. Frost, *Gray Hat Hacking: The Ethical Hacker's Handbook*, 6th ed., New York, NY, USA: McGraw-Hill, 2022.
[46] R. Gandhi, H. Siy, and Y. Wu, "Lightweight formal models of software weaknesses," in *Proc. 1st FME Workshop Formal Methods Softw. Eng. (FormaliSE)*, May 2013, pp. 50–56, doi: 10.1109/FormaliSE.2013.6612277.
[47] T. Mikolov, K. Chen, G. S. Corrado, and J. B. Dean, "Efficient estimation of word representations in vector space," in *Workshop Proc. Int. Conf. Learn. Represent. (ICLR)*, Jan. 2013, pp. 1–14.
[48] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, J. Burstein, C. Doran, and T. Solorio, Eds., 2019, pp. 4171–4186, doi: 10.18653/v1/n19-1423.
[49] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.
[50] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," OpenAI, San Francisco, CA, USA, Tech. Rep., 2018. [Online]. Available: https://www.mikecaptain.com/resources/pdf/GPT-1.pdf
[51] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
[52] A. Sherstinsky, "Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network," *Phys. D, Nonlinear Phenomena*, vol. 404, Mar. 2020, Art. no. 132306, doi: 10.1016/j.physd.2019.132306.
[53] *Software Assurance Reference Dataset (SARD)*, Nat. Inst. Standards Technol. (NIST), Gaithersburg, MD, USA, 2023.
[54] Z. Li, S. Xing, L. Yu, H. Li, F. Zhou, G. Yin, X. Tang, and Z. Wang, "Binary program vulnerability mining based on neural network," *Comput., Mater. Continua*, vol. 78, no. 2, pp. 1861–1879, 2024, doi: 10.32604/cmc.2023.046595.
[55] D. Brumley, I. Jager, T. Avgerinos, and E. J. Schwartz, "BAP: A binary analysis platform," in *Proc. Int. Conf. Comput. Aided Verification*, Snowbird, UT, USA, Jul. 2011, pp. 463–469, doi: 10.1007/978-3-642-22110-1_37.
[56] N. Naus, F. Verbeek, S. Atla, and B. Ravindran, "Poster: Formally verified binary lifting to P-code," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Dec. 2024, pp. 4973–4975, doi: 10.1145/3658644.3691386.
[57] C. Casinghino, J. T. Paasch, C. Roux, J. Altidor, M. Dixon, and D. Jamner, "Using binary analysis frameworks: The case for BAP and angr," in *Proc. NASA Formal Methods Symp.*, Houston, TX, USA, Jan. 2019, pp. 123–129, doi: 10.1007/978-3-030-20652-9_8.
[58] J. Kroustek, P. Matula, and P. Zemek, "Retdec: An open-source machine-code decompiler," in *Proc. Botnet Fighting Conf.*, 2017, pp. 1–15.
[59] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. Salakhutdinov, and Q. V. Le, "XLNet: Generalized autoregressive pretraining for language understanding," 2019, *arXiv:1906.08237*.
[60] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A lite BERT for self-supervised learning of language representations," 2019, *arXiv:1909.11942*.
[61] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, "ELECTRA: Pre-training text encoders as discriminators rather than generators," 2020, *arXiv:2003.10555*.
[62] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with disentangled attention," 2020, *arXiv:2006.03654*.
[63] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," 2020, *arXiv:2004.05150*.
[64] M. Zaheer, G. Guruganesh, A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed, "Big bird: Transformers for longer sequences," 2020, *arXiv:2007.14062*.
[65] S. Guido and A. C. Mueller, *Introduction to Machine Learning with Python*. Sebastopol, CA, USA: O'Reilly Media, Oct. 2016.
[66] A. Géron, *Hands-on Machine Learning with Scikit-Learn, Keras, and Tensorflow: Concepts, Tools and Techniques to Build Intelligent Systems*. Beijing, China: O'Reilly, 2019.
[67] I.-s. Kang, B.-s. Kim, S.-b. Park, G.-w. Yoon, J.-h. Cho, and H.-j. Suh, "Research on static analysis-based secure coding tools using ai and llvm," in *Proc. Annu. Conf. KIPS*, 2024, pp. 831–832.
[68] D. Arp et al., "Dos and don'ts of machine learning in computer security," in *Proc. USENIX Secur. Symp.*, 2022, pp. 3971–3988.
[69] G. A. McCully, J. D. Hastings, and S. Xu, "Impact of data snooping on deep learning models for locating vulnerabilities in lifted code," in *Proc. Int. Symp. Intell. Comput. Netw. (ISICN)*, Cham, Switzerland: Springer, 2025, pp. 1–14.
[70] J. Howard and S. Ruder, "Universal language model fine-tuning for text classification," in *Proc. 56th Annu. Meeting Assoc. Comput. Linguistics*, 2018, pp. 328–339, doi: 10.18653/v1/p18-1031.
[71] A. Shestov, R. Levichev, R. Mussabayev, E. Maslov, A. Cheshkov, and P. Zadorozhny, "Finetuning large language models for vulnerability detection," 2024, *arXiv:2401.17010*.
[72] R. Liu, Y. Wang, H. Xu, B. Liu, J. Sun, Z. Guo, and W. Ma, "Source code vulnerability detection: Combining code language models and code property graphs," 2024, *arXiv:2404.14719*.
[73] S. Kim, J. Choi, M. E. Ahmed, S. Nepal, and H. Kim, "VulDeBERT: A vulnerability detection system using BERT," in *Proc. IEEE Int. Symp. Softw. Rel. Eng. Workshops (ISSREW)*, Oct. 2022, pp. 69–74, doi: 10.1109/ISSREW55968.2022.00042.
[74] J. Su and Y. Wu, "Optimizing pre-trained language models for efficient vulnerability detection in code snippets," in *Proc. 9th Int. Conf. Comput. Commun. (ICCC)*, Dec. 2023, pp. 2139–2143, doi: 10.1109/iccc59590.2023.10507456.
[75] N. Ziems and S. Wu, "Security vulnerability detection using deep learning natural language processing," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, May 2021, pp. 1–6, doi: 10.1109/INFOCOMWKSHPS51825.2021.9484500.
[76] K. Singh, S. S. Grover, and R. K. Kumar, "Cyber security vulnerability detection using natural language processing," in *Proc. IEEE World AI IoT Congr. (AIIoT)*, Jun. 2022, pp. 174–178, doi: 10.1109/AIIoT54504.2022.9817336.
[77] C. Thapa, S. I. Jang, M. E. Ahmed, S. Camtepe, J. Pieprzyk, and S. Nepal, "Transformer-based language models for software vulnerability detection," in *Proc. 38th Annu. Comput. Secur. Appl. Conf.*, Dec. 2022, pp. 481–496, doi: 10.1145/3564625.3567985.
[78] N. T. Islam, G. De La Torre Parra, D. Manuel, E. Bou-Harb, and P. Najafirad, "An unbiased transformer source code learning with semantic vulnerability graph," in *Proc. IEEE 8th Eur. Symp. Secur. Privacy (EuroS&P)*, Jul. 2023, pp. 144–159, doi: 10.1109/eurosp57164.2023.00018.
[79] M. Omar and S. Shiaeles, "VulDetect: A novel technique for detecting software vulnerabilities using language models," in *Proc. IEEE Int. Conf. Cyber Secur. Resilience (CSR)*, Jul. 2023, pp. 105–110, doi: 10.1109/csr57506.2023.10224924.
[80] C. Zhang, H. Liu, J. Zeng, K. Yang, Y. Li, and H. Li, "Prompt-enhanced software vulnerability detection using ChatGPT," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng., Companion*, Apr. 2024, p. 276, doi: 10.1145/3639478.3643065.

[81] X. Du, M. Wen, J. Zhu, Z. Xie, B. Ji, H. Liu, X. Shi, and H. Jin, "Generalization-enhanced code vulnerability detection via multi-task instruction fine-tuning," in *Proc. Findings Assoc. Comput. Linguistics*, 2024, pp. 10507–10521, doi: 10.18653/v1/2024.findings-acl.625.

[82] Z. Wang, G. Li, J. Li, Y. Xiong, J. Li, M. Yan, and Z. Jin, "M2CVD: Enhancing vulnerability semantic through multi-model collaboration for code vulnerability detection," 2024, *arXiv:2406.05940*.

[83] Y. Liu, L. Gao, M. Yang, Y. Xie, P. Chen, X. Zhang, and W. Chen, "VulDetectBench: Evaluating the deep capability of vulnerability detection with large language models," 2024, *arXiv:2406.07595*.

[84] A. Khare, S. Dutta, Z. Li, A. Solko-Breslin, R. Alur, and M. Naik, "Understanding the effectiveness of large language models in detecting security vulnerabilities," 2023, *arXiv:2311.16169*.

[85] J. Zhang, C. Wang, A. Li, W. Sun, C. Zhang, W. Ma, and Y. Liu, "An empirical study of automated vulnerability localization with large language models," 2024, *arXiv:2404.00287*.

[86] X. Zhou, T. Zhang, and D. Lo, "Large language model for vulnerability detection: Emerging results and future directions," in *Proc. ACM/IEEE 44th Int. Conf. Softw. Eng., New Ideas Emerg. Results*, Apr. 2024, pp. 47–51, doi: 10.1145/3639476.3639762.

[87] A. Jones and M. Omar, "Codesentry: Revolutionizing real-time software vulnerability detection with optimized GPT framework," *Land Forces Acad. Rev.*, vol. 29, no. 1, pp. 98–107, Mar. 2024, doi: 10.2478/raft-2024-0010.

[88] Z. Liu, Q. Liao, W. Gu, and C. Gao, "Software vulnerability detection with GPT and in-context learning," in *Proc. 8th Int. Conf. Data Sci. Cyberspace (DSC)*, Aug. 2023, pp. 229–236, doi: 10.1109/dsc59305.2023.00041.

[89] M. D. Purba, A. Ghosh, B. J. Radford, and B. Chu, "Software vulnerability detection using large language models," in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng. Workshops (ISSREW)*, Oct. 2023, pp. 112–119, doi: 10.1109/issrew60843.2023.00058.

[90] A. Bahaa, A. E.-R. Kamal, H. Fahmy, and A. S. Ghoneim, "DB-CBIL: A DistilBert-based transformer hybrid model using CNN and BiLSTM for software vulnerability detection," *IEEE Access*, vol. 12, pp. 64446–64460, 2024, doi: 10.1109/ACCESS.2024.3396410.

[91] N. Saji Mathews, Y. Brus, Y. Aafer, M. Nagappan, and S. McIntosh, "LLbezpeky: Leveraging large language models for vulnerability detection," 2024, *arXiv:2401.01269*.

[92] Y. J. Lee, S.-H. Choi, C. Kim, S.-H. Lim, and K.-W. Park, "Learning binary code with deep learning to detect software weakness," in *Proc. KSII 9th Int. Conf. Internet (ICONI) Symp.*, Dec. 2017, pp. 1–5.

[93] S. Heidbrink, K. N. Rodhouse, and D. M. Dunlavy, "Multimodal deep learning for flaw detection in software programs," 2020, *arXiv:2009.04549*.

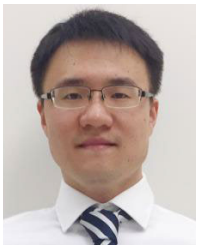
[94] B. M. Padmanabhuni and H. B. K. Tan, "Buffer overflow vulnerability prediction from x86 executables using static analysis and machine learning," in *Proc. IEEE 39th Annu. Comput. Softw. Appl. Conf.*, vol. 2, Jul. 2015, pp. 450–459, doi: 10.1109/COMPSAC.2015.78.

GARY A. MCCULLY (Member, IEEE) was born in Elyria, OH, USA. He received the B.S. degree (cum laude) from Indiana Wesleyan University, in 2014, the M.S. degree (Hons.) in information assurance and cybersecurity from Capella University, in 2020, and the Ph.D. degree in cyber operations from Dakota State University, in 2025.

He has over 15 years of experience in the cybersecurity field and is currently a Senior Offensive Security Engineer with PayPal. His career has included roles focused on red and purple teaming, penetration testing, and application testing. His research interests include reverse engineering, exploit development, and the application of machine learning to vulnerability detection. He has earned certifications, such as the Certified Information Systems Security Professional (CISSP), the Exploit Researcher and Advanced Penetration Tester (GXPN), the GIAC Cloud Penetration Testing (GCPN), the Certified Ethical Hacker (CEH), and the Offensive Security Certified Professional (OSCP).

Dr. McCully is a member of the Association for Computing Machinery (ACM).

JOHN D. HASTINGS received the B.S. (Hons.), M.S., and Ph.D. degrees in computer science from the University of Wyoming. He has several years of industry experience as an AI/ML Software Engineer, the Team Leader, and an AI/ML Consultant. He is currently a Professor with The Beacom College of Computer and Cyber Sciences, Dakota State University. He has published numerous peer-reviewed papers in a variety of prestigious conferences and journals. His research interests include machine learning and artificial intelligence, cybersecurity, ecological and environmental applications of AI, gamification in CS and cybersecurity education, and AI in games. He has won multiple awards related to his research, including best paper awards and the Innovative Applications of AI Award.



SHENGJIE XU (Senior Member, IEEE) received the B.E. degree in information security, in 2012, the M.S. degree in telecommunications from the University of Pittsburgh, in 2014, and the Ph.D. degree in computer engineering from the University of Nebraska-Lincoln, in 2019.

He has been a tenure-track Assistant Professor with The University of Arizona (U of A), since August 2024, and a Co-Principal Investigator (Co-PI) of the Google-Funded Arizona Cybersecurity Clinic. Before joining U of A, he was a tenure-track Assistant Professor with San Diego State University, from 2022 to 2024, and Dakota State University, from 2019 to 2022. His research interests include cybersecurity, cyber AI, zero trust, cryptography, secure AI, and graph modeling. He is the author of one book and more than 40 articles. He is the lead author of the book *Cybersecurity in Intelligent Networking Systems* (John Wiley/IEEE Press, 2023). He holds multiple professional certifications in cybersecurity and computer networking. He received grants and contracts in research and educational projects from U.S. National Science Foundation (NSF) and other organizations.

Dr. Xu was a recipient of the Exemplary Editor Award of 2024 from *IEEE Wireless Communications Magazine* in 2025, the Outstanding Faculty Award from the Fowler College of Business at San Diego State University in 2024, the Institution of Engineering and Technology (IET) Journals Premium Award for Best Paper in 2020, the Milton E. Mohr Graduate Fellowship Award from the College of Engineering at the University of Nebraska-Lincoln in 2017, and the Best Poster Award from the International Conference on Design of Reliable Communication Networks (DRCN) in 2015. He currently serves as a Technical Editor for *IEEE Wireless Communications Magazine*, an Associate Editor for IEEE Access, an Editor for the *International Journal of Sensor Networks*, and the Technical Program Committee (TPC) Vice Chair for the International Symposium on Intelligent Computing and Networking (ISICN) 2025. He served as a Proposal Panel Reviewer for the NSF.